



Documento a cura di Simone Remoli

Prefazione.

Il presente documento si propone di fornire una conoscenza dei principi fondamentali

relativi alla **progettazione di un database**. L'obiettivo ultimo è consentire una

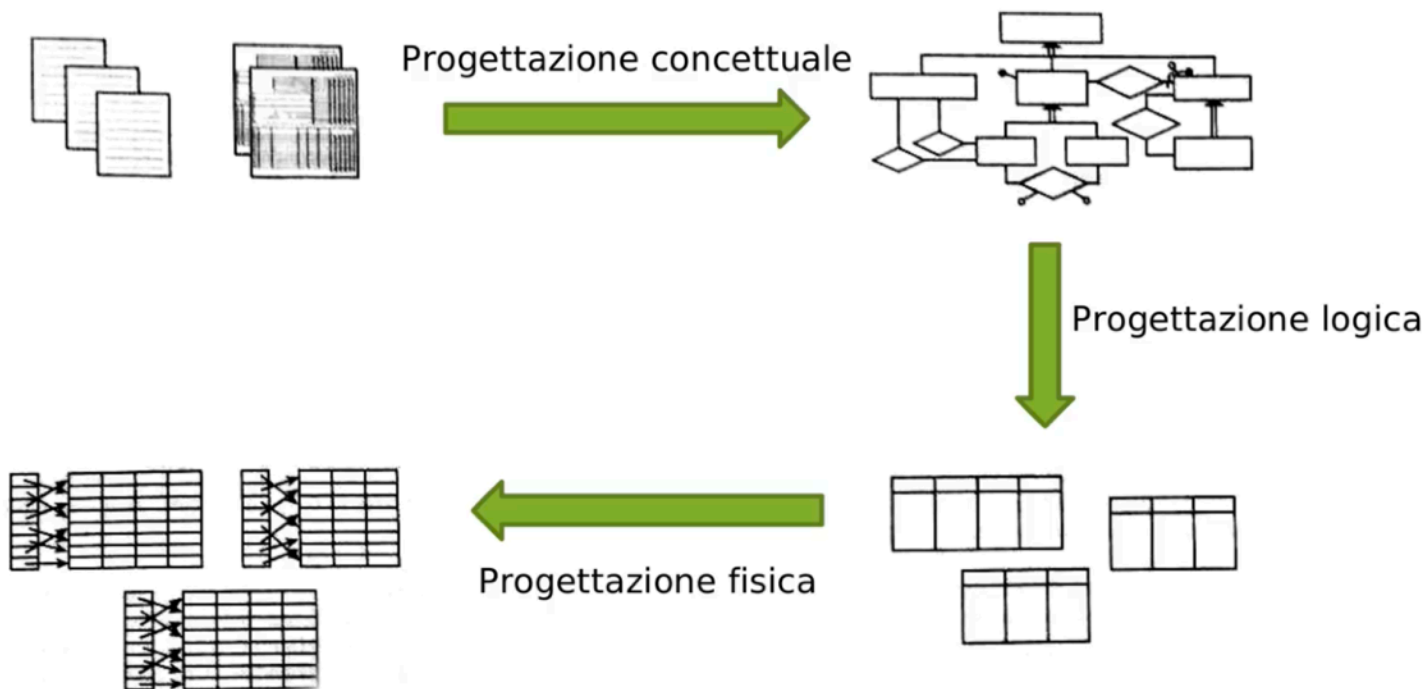
padronanza teorica e pratica degli strumenti

necessari per affrontare con successo l'esame. A tal fine, verrà trattato in maniera

esaustiva l'argomento di rilievo molto importante: il **modello concettuale**.

Buona lettura.

Le fasi di progetto di una Base di Dati.

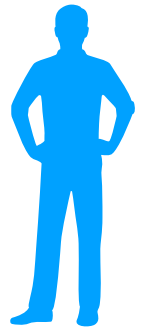


L'immagine mostra le fasi di progettazione di una base di dati, suddivise in tre livelli:

1. **Progettazione concettuale** - Definisce la struttura dei dati a livello astratto, spesso rappresentata con diagrammi Entità-Relazione (E-R).
2. **Progettazione logica** - Trasforma il modello concettuale in un modello logico adatto a un sistema di gestione di database (DBMS), come un modello relazionale con tabelle e relazioni.
3. **Progettazione fisica** - Ottimizza l'archiviazione e l'accesso ai dati nel DBMS, considerando strutture di indicizzazione, partizionamento e gestione della memoria.

Prima di scendere nel dettaglio: attenti al cliente!

Nel contesto reale della progettazione di una **base di dati**, il cliente rappresenta spesso un'**entità imprevedibile**, priva di una chiara visione dei propri bisogni ma convinta di possedere una conoscenza approfondita. Le sue richieste risultano spesso **contraddittorie, imprecise e incoerenti**, rendendo l'**analisi dei requisiti** una delle sfide più complesse. L'esperienza con il cliente è un'esperienza che si rivela più simile a una **seduta di psicoterapia** che a un'analisi tecnica.



 **Cliente:** "Voglio questa funzionalità, però... non sono sicuro se mi serve realmente."

 **Analista:** "Quindi la implementiamo o no?"

 **Cliente:** "Beh, voi che ne pensate? Ha senso?"

 **Analista:** "No, ma ormai siamo in questa conversazione esistenziale, quindi..."

 **Cliente:** "Per l'ultima parte devo pensarci... però mettimi questa intanto."

 **Analista:** "Ok, se mettiamo questa, allora dovremmo aggiungere anche quest'altra funzionalità per mantenere coerenza tra i dati."

 **Cliente:** "No, quella no... oddio, forse sì... magari a breve mi interfaccio con chi di dovere e vi fornisco una specifica giusta."

 **Analista:** "Ok, quindi non la mettiamo?"

 **Cliente:** "Boh, intanto iniziate, poi vediamo."

Conclusione: Hai sviluppato un'ansia da progetto senza neanche aver scritto una riga di codice.

Insomma, **raccogliere i requisiti è un'arte**, un misto tra psicologia, negoziazione e capacità di non perdere la sanità mentale. E poi, quando tutto sembra finalmente chiaro... BAM! Il cliente torna dritta sta hit **"Ho parlato con chi di dovere e dobbiamo rifare tutto"**.

PROGETTAZIONE CONCETTUALE.

La **progettazione concettuale** nasce come strumento indispensabile per **domare la "bestia" del cliente**, trasformando richieste vaghe, contraddittorie e spesso incoerenti in una struttura chiara, organizzata e formalmente definita. Questo è compito dell'**analista**.

Essa funge da **ponte tra il linguaggio tecnico e quello del cliente**, permettendo di tradurre esigenze poco strutturate in un modello solido e comprensibile, ponendo così le basi per una progettazione efficace e coerente della base di dati. Il **modello concettuale** utilizza principalmente il **linguaggio grafico del modello Entità-Relazione (E-R)**, ideato da **Peter Chen** nel 1976. Questo linguaggio è rappresentato mediante un **diagramma E-R**, che permette di descrivere la struttura della base di dati in modo **visivo e intuitivo**, indipendentemente dal DBMS che verrà utilizzato.

PROGETTAZIONE LOGICA.

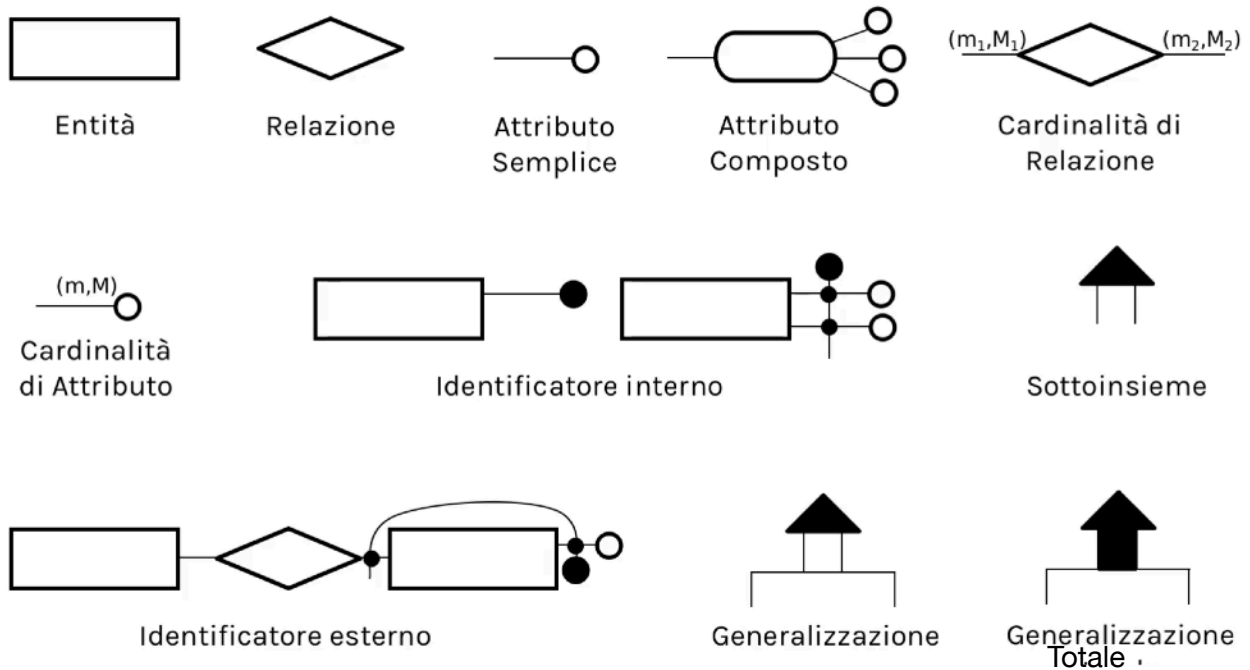
La **progettazione logica** è la fase intermedia del processo di progettazione di una **base di dati**, in cui il **modello concettuale** viene tradotto in una rappresentazione più strutturata e adatta all'implementazione su un **DBMS** (Database Management System). In questa fase, il modello **entità-relazione (E-R)** viene trasformato in un **modello relazionale**, definendo tabelle, attributi, tipi di dati, vincoli di integrità e relazioni tra le entità.

PROGETTAZIONE FISICA.

La **progettazione fisica** è la fase finale della progettazione di una base di dati, in cui il **modello logico** viene tradotto in una struttura concreta, ottimizzata per l'archiviazione e l'accesso efficiente ai dati su un **DBMS specifico**.

Notazione di Chen nel Modello Entità-Relazione (E-R)

La **notazione di Chen**, introdotta da **Peter Chen** nel 1976, è un metodo grafico per rappresentare il **modello Entità-Relazione (E-R)**, utile nella progettazione concettuale delle basi di dati.



Entità → Rettangoli

Rappresentano oggetti reali o concetti della base di dati (es. *Studente*, *Corso*).

Relazioni → Rombi

Collegano due o più entità (es. *Studente segue Corso*).

Attributi → Ovali

Descrivono proprietà delle entità o delle relazioni (es. *Nome*, *Matricola*).

Attributo Composto (Ovale con più sotto-attributi)

Un attributo che può essere suddiviso in più componenti, ma collegato a una relazione. (es. *Nome* suddiviso in *Nome* e *Cognome*).

Cardinalità di Relazione (Annotazione accanto alla relazione)

Indica il numero minimo e massimo di istanze coinvolte nella relazione (es. $(1,N)$).

Cardinalità di Attributo (Annotazione accanto a un attributo)

Specifica il numero minimo e massimo di valori che un attributo può assumere.

Identificatore Interno (punto nero su)

Chiave primaria che identifica univocamente l'entità.

Sottoinsieme (Freccia vuota verso l'alto)

Indica che un'entità è un sottoinsieme di un'altra entità.

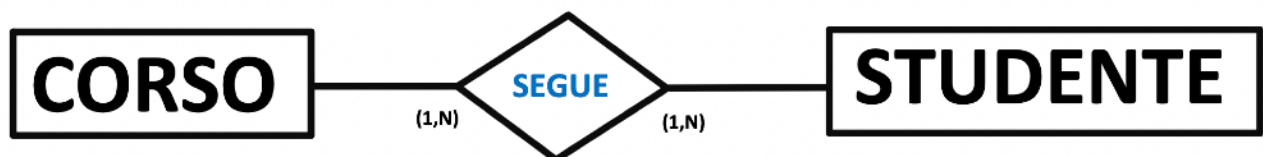
Generalizzazione (Freccia piena verso l'alto)

Un'entità generale può essere specializzata in sottoentità (es. *Veicolo* → *Auto*, *Moto*).

Esempio di Notazione Chen

Caso: Studenti e Corsi

- **Entità:** *Studente*, *Corso*
- **Relazione:** *Segue*
- **Attributi:** *Nome*, *Matricola* (chiave primaria), *CFU* (per il corso)



Uno *Studente* può seguire **più** corsi $(1:N)$.
Quindi può seguire da 1 a infiniti corsi $(1,N)$.

Un *Corso* può essere seguito da **più** studenti $(1:N)$.
Quindi può essere seguito da 1 a infiniti studenti $(1,N)$.

Nella notazione delle **cardinalità** nel modello **Entità-Relazione (E-R)**, la coppia (x,y) indica il **numero minimo e massimo di istanze** che un'entità può avere in una relazione.

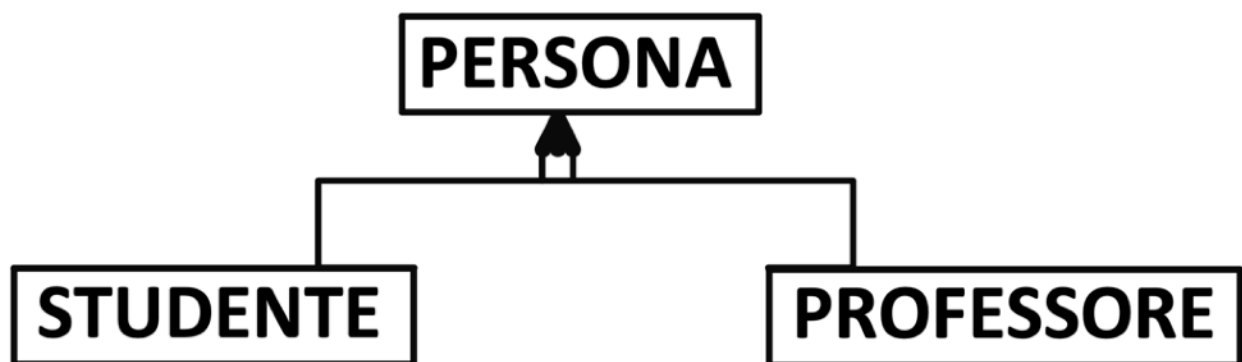
Il **primo valore (x)** indica il **minimo** numero di istanze richieste.

Il **secondo valore** (**y**) indica il **massimo** numero di istanze possibili.

Se invece un corso potrebbe non avere studenti iscritti, allora la cardinalità diventa (0, N).

Il corso partecipa alla relazione segue con cardinalità (1,N) e lo studente partecipa alla relazione segue con cardinalità (1,N).

LA SPECIALIZZAZIONE/ GENERALIZZAZIONE PARZIALE.



Se, ad esempio, scrivo in questo modo, sto indicando che l'entità **Persona** viene suddivisa nelle sotto-entità **Studente** e **Professore**.

Di conseguenza, il modello rappresenta sia gli **studenti**, sia i **professori**, sia un insieme di persone che non rientrano in nessuna delle due categorie e questo implica che esistono persone che non sono né studenti né professori.

Ci sono delle persone, alcune di queste sono studenti e alcune di queste sono professori.

LA SPECIALIZZAZIONE/ GENERALIZZAZIONE TOTALE.

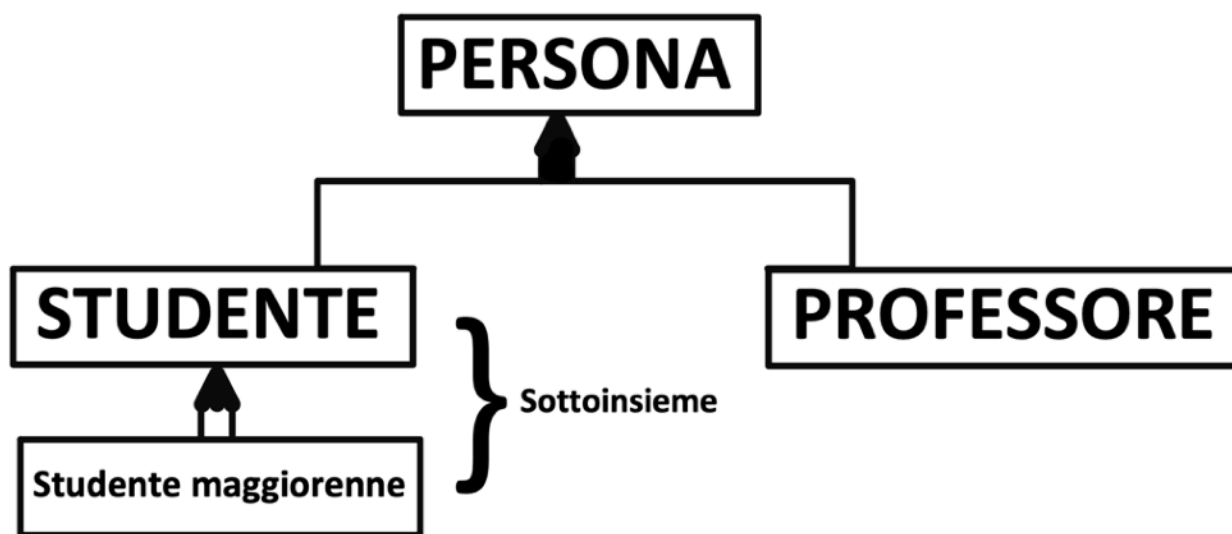
Tutte le persone sono o studenti o professori: nessun'altra categoria.



Però, se prendiamo come minimondo da esaminare le scuole dalle elementari fino alle superiori, allora è giusto considerare **studenti maggiorenni e studenti minorenni**.

Possiamo specializzare lo studente in studente maggiorenne con la **generalizzazione parziale** poichè alcuni studenti sono maggiorenni.

Nel caso in cui la generalizzazione parziale è tale per cui ha solamente una sotto-entità la struttura si chiama **sottoinsieme**.



Un sottoinsieme degli studenti del minimondo è maggiorenne.

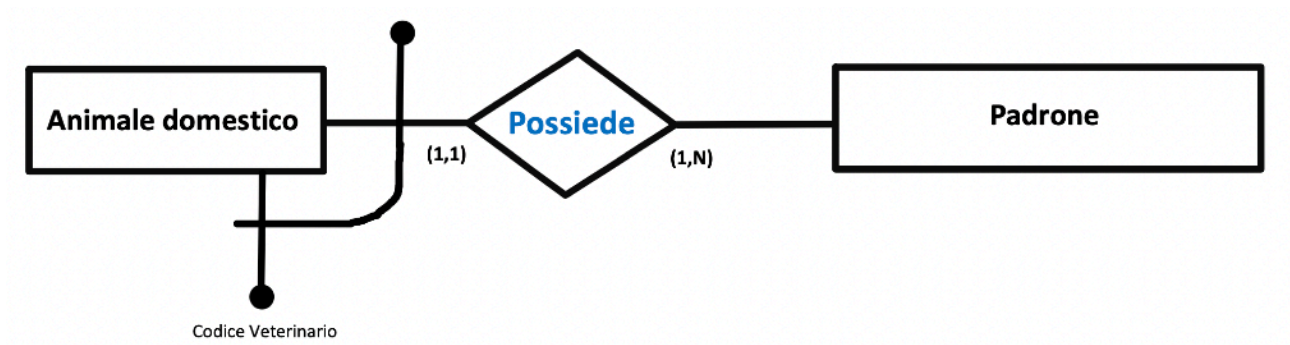
ENTITÀ DEBOLE.

L'entità debole non può esistere se non esiste un'entità forte cui si partecipa.

Non ha senso che l'animale domestico esista senza un padrone, per questo motivo animale domestico è un'entità debole.

Non è sufficiente identificare l'animale domestico con il padrone perché il padrone può possedere più animali domestici.

Quindi l'animale domestico è identificato dal codice veterinario e il padrone che lo possiede.



Non ci interessa rappresentare degli animali domestici se questi non sono legati al loro padrone.

“Animale domestico” è un’entità debole verso l’entità forte “Padrone”.

LINGUAGGIO NATURALE.

Quando serve occorre specificare in linguaggio naturale determinati passaggi seguendo un pattern strutturato:

Vincoli di integrità: <concetto> deve/non deve <expr sui concetti>

Derivazioni: <concetto> si ottiene <operazione sui concetti>.

Dizionario dei Dati: è una documentazione di supporto (a corredo) del diagramma ER dove, per ciascuna entità, forniamo una piccola definizione in linguaggio naturale, elenchiamo tutti gli attributi e descriviamo tutte le relazioni con cui questa entità partecipa con la rispettiva cardinalità.

Teorema sotto-entità.

Teorema: TUTTE le sotto-entità ereditano le caratteristiche dell’entità padre.

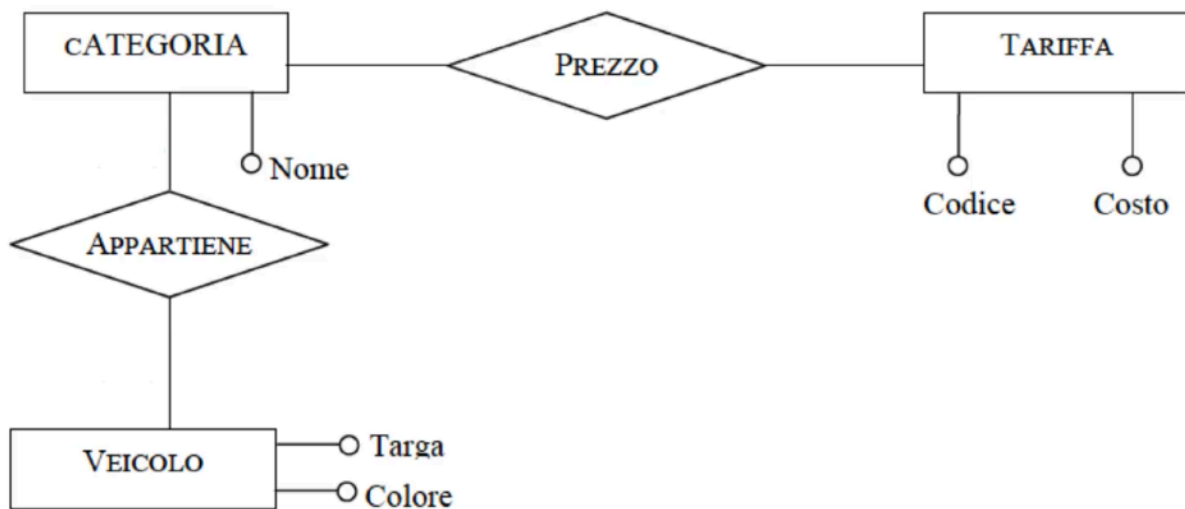
ESERCIZI MODELLO E/R.

Esercizio 1.

Una agenzia di noleggio di autovetture ha un parco macchine ognuna delle quali ha una targa, un colore e fa parte di una categoria; per ogni categoria c'è una tariffa di noleggio.

Progettare lo schema E/R.

Soluzione.

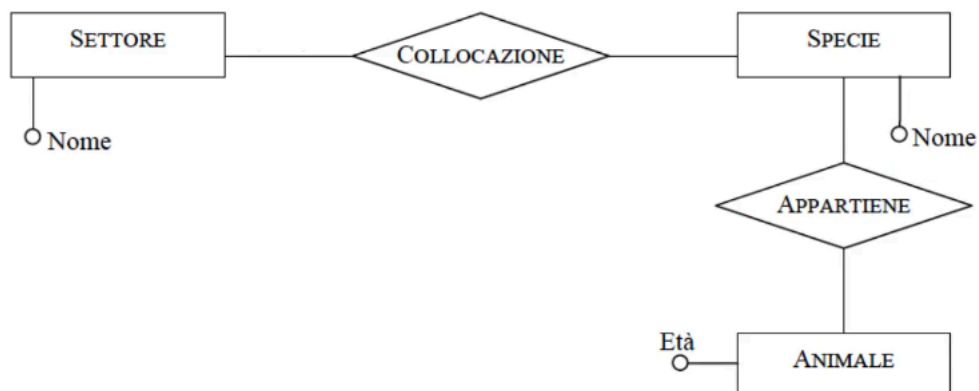


Esercizio 2.

In un giardino zoologico ci sono degli animali appartenenti a una specie e aventi una certa età; ogni specie è localizzata in un settore (avente un nome) dello zoo.

Progettare lo schema E/R.

Soluzione.



(**Sto omettendo volontariamente le cardinalità**
poichè l'obiettivo è prendere confidenza con il modello
ER).

Man mano le aggiungo (esercizi successivi).

Però se ho due animali della stessa specie e della stessa
età come li distinguo? Bisognerebbe fare delle piccole
modifiche, ma intanto lo schema prendiamocelo buono.

Esercizio 3.

Gli impiegati di una azienda si dividono in **dirigenti**,
programmatori, , **capi progetto e segretari**.
Ci sono analisti (alcuni analisti, non tutti) che sono
anche programmatori.

I capi progetto **devono essere** dirigenti.

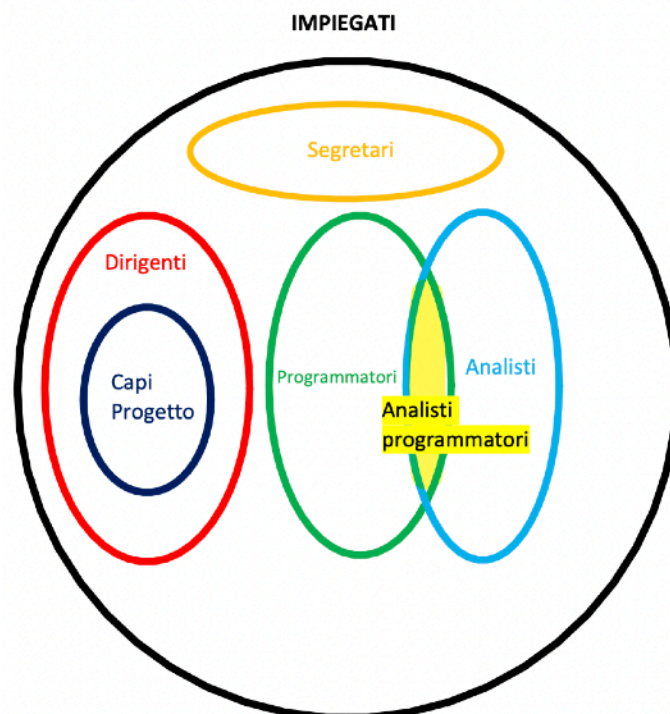
Gli impiegati hanno un codice, un nome e un cognome.
Ogni categoria di impiegato ha un proprio stipendio base.
Ogni impiegato, tranne i dirigenti, ha un orario di
lavoro.

Progettare lo schema E/R.

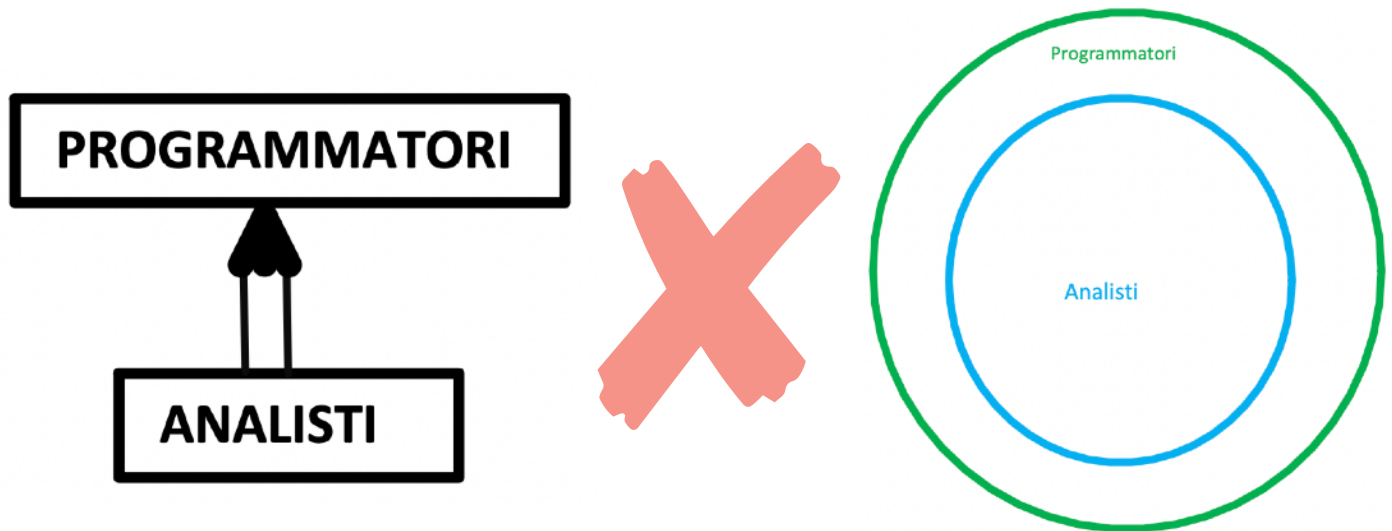
Soluzione.

Nota: in questi casi si può sbagliare la costruzione del
modello E/R poichè si potrebbe pensare alla struttura di
un organigramma aziendale, inserendo come entità generale
i dirigenti, _.

Dal punto di vista insiemistico lo schema è il seguente:



La fase struggente di questa specifica è il tipo di relazione che vige tra **Programmatori** e **Analisti**.
Lo schema insiemistico sopra riportato è quello corretto, però un tipico **errore fuorviante** per descrivere questa relazione può essere il seguente:



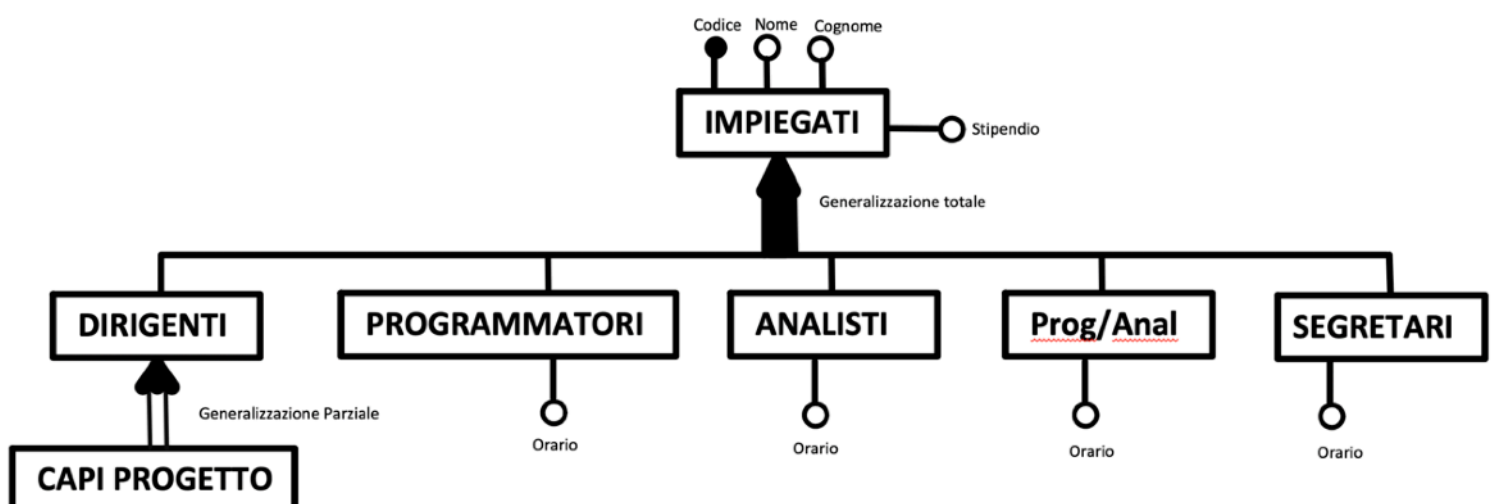
“Alcuni programmatori sono analisti”

Ma

“Tutti gli analisti sono programmatori”

E gli analisti non programmatori? Questa situazione non è catturata.

Soluzione: aggiungo una nuova entità **“Programmatori/Analisti”** che mi cattura **l'intersezione** tra **Programmatori** e **Analisti**.



Poiché ogni categoria di impiegato è associata a uno **stipendio base**, l'attributo **stipendio** viene definito direttamente nell'entità generale "**Impiegati**", permettendo così alle sotto-entità **Dirigenti, Programmatori, Analisti, Prog/Anal e Segretari** di ereditarlo automaticamente.

L'attributo orario di lavoro, invece, non può essere inserito nell'entità generale "Impiegati", poiché, se così fosse, verrebbe ereditato anche dall'entità **Dirigenti**. Tuttavia, la specifica indica chiaramente che tale attributo deve appartenere a tutti gli impiegati, ad eccezione dei dirigenti. Pertanto, è necessario gestirlo a un livello inferiore della gerarchia, evitando che venga assegnato in modo non appropriato.

Nota: ereditare l'attributo non significa ereditarne anche il valore, ma solo l'attributo: nella sottoclasse potrà essere valorizzato in maniera differente.

ORA COMINCIAMO L'IMMERSIONE NEL DETTAGLIO.

Differenza tra Notazione Chen e Notazione UML nella Modellazione E/R.

La notazione Chen e la notazione UML sono due approcci differenti per la modellazione concettuale delle basi di dati. Entrambe servono a rappresentare entità, relazioni e attributi, ma si differenziano per il livello di astrazione, l'applicabilità e la chiarezza visiva.

Struttura della Notazione UML

1. **Entità** → Rappresentate come **classi** con attributi in rettangoli.
2. **Relazioni** → Linee con **cardinalità** (1:N, N:N).
3. **Attributi** → Inseriti all'interno delle classi.
4. **Chiavi primarie** → Indicate come (PK) accanto agli attributi.
5. **Chiavi esterne** → Indicate come (FK) accanto agli attributi.

In questa trattazione adotterò entrambe le notazioni.

Pur riconoscendo che la notazione di Chen offre una rappresentazione chiara e intuitiva delle **entità, relazioni e attributi**, UML consente di rappresentare con maggiore immediatezza la struttura logica del database, semplificando la transizione verso la realizzazione effettiva dello schema relazionale. Pertanto, tutti i diagrammi e le modellazioni presenti in questa trattazione seguiranno entrambe le notazioni in maniera intercambiabile.

Forma Intensionale ed Estensionale delle Tabelle nei Database.

Nel **modello relazionale** delle basi di dati, una tabella può essere descritta da due punti di vista fondamentali:

- **Forma intensionale (intensione)** → Descrive la **struttura** della tabella (schema, attributi, tipi di dati).

Esempio:

`Studente`(Matricola, Nome, Cognome, DataNascita, CorsoDiLaurea).

- **Forma estensionale (estensione)** → Descrive il **contenuto** della tabella (tuple, valori effettivi).

Matricola	Nome	Cognome	DataNascita	CorsoDiLaurea
1001	Luca	Bianchi	2000-05-15	Informatica
1002	Maria	Rossi	1999-09-20	Ingegneria
1003	Marco	Verdi	2001-01-10	Economia

Cominciamo a costruire dei database completi.

Nel concreto:

Il **modello concettuale** (o **modello E/R**) rappresenta in modo astratto e formale la realtà di riferimento dei dati e le loro reciproche relazioni attraverso uno schema grafico.

Input: Requisiti cliente.

Output: E/R schema.

Il **modello logico** viene ottenuto dal modello concettuale applicando le "**regole di derivazione**".

Input: E/R.

Output: Schema logico rappresentato dalle tabelle in forma intensionale.

A partire dal modello concettuale vengono applicate le regole di derivazione in modo da ottenere un modello logico idoneo.

ATTRIBUTI.

Gli attributi definiti all'interno di un'entità devono essere **elementari e atomici**, ovvero non scomponibili in ulteriori sotto-attributi.

Inoltre, non è consentita la definizione di **attributi derivati**, ovvero quegli attributi che possono essere ottenuti mediante un'elaborazione sui dati già presenti nel sistema.

Ad esempio, l'**età** di una persona non deve essere esplicitamente memorizzata, poiché può essere calcolata dinamicamente a partire dalla **data di nascita** e dalla **data corrente**.

Questo principio garantisce la **coerenza e l'integrità dei dati**, evitando ridondanze e possibili incongruenze nel database.

LA CHIAVE PRIMARIA (Primary Key PK).

La **chiave primaria** è un attributo o un insieme minimale di attributi (PK composta) che permette di distinguere le istanze (tuple) di una stessa entità.

Ogni PK composta è chiamata "Primary Key Parziale" ed è etichettata aggiungendo "PPK" dopo il nome dell'attributo. In sembianza grafica, ogni campo che fa parte della PK è sottolineato.

La chiave primaria può essere composta da più colonne, creando una chiave primaria composta.

I valori della chiave primaria non possono essere NULL (vincolo di NOT NULL).

I valori della chiave primaria devono essere univoci per ogni riga della tabella.

NOTA: Se una chiave primaria è composta da più attributi, quando viene referenziata in un'altra tabella, tutti gli attributi devono diventare chiavi esterne.

Le FOREIGN KEY devono sempre avere gli stessi tipi di dati e dimensioni dei rispettivi attributi nella tabella di riferimento. Se una tabella referencia una PK composta, la FK dovrà avere tutti gli attributi della PK.

Questo garantisce l'integrità referenziale e mantiene la relazione tra le entità.

LA CHIAVE ESTERNA (Foreign Key FK).

Una chiave esterna (FOREIGN KEY) è un vincolo utilizzato nei database relazionali per mantenere l'integrità referenziale tra due tabelle.

Una chiave esterna è un attributo (o un insieme di attributi) in una tabella che fa riferimento a una **chiave primaria (PRIMARY KEY)** in un'altra tabella.

```
CREATE TABLE NomeTabella (  
  
    NomeAttributo INT,  
  
    FOREIGN KEY (NomeAttributo) REFERENCES  
    NomeTabellaReferenziata(NomeChiavePrimaria));
```

È possibile avere attributi che sono contemporaneamente chiavi primarie (PRIMARY KEY) e chiavi esterne (FOREIGN KEY).

```
CREATE TABLE Esame (  
    ID_Studente INT,  
    ID_Corso INT,  
    Data_Esame DATE,  
    Voto INT,  
    PRIMARY KEY (ID_Studente, ID_Corso, Data_Esame)  
);
```

```
CREATE TABLE Risultato (  
    ID_Studente INT,  
    ID_Corso INT,  
    Data_Esame DATE,  
    Note VARCHAR(255),  
    PRIMARY KEY (ID_Studente, ID_Corso, Data_Esame), -- PK  
composta  
    FOREIGN KEY (ID_Studente, ID_Corso, Data_Esame) REFERENCES  
    Esame(ID_Studente, ID_Corso, Data_Esame)  
);
```

se il lato "N" ha esattamente un'istanza per ogni istanza del lato "1", la FK può essere una primary key.

È anche possibile che una tabella abbia una chiave primaria composta (PK) e altri attributi separati che sono chiavi esterne (FK).

```
CREATE TABLE Cliente (  
  
    ID_Cliente INT PRIMARY KEY,  
  
    Nome VARCHAR(100)
```



```
);
```

```
CREATE TABLE Prodotto (  
    ID_Prodotto INT PRIMARY KEY,  
    Nome VARCHAR(100)  
);
```

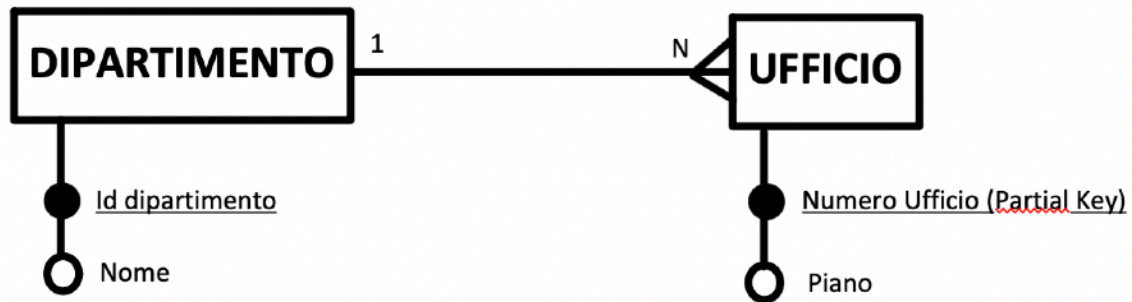
```
CREATE TABLE Fattura (  
    ID_Fattura INT PRIMARY KEY, -- PK non è FK  
    ID_Cliente INT,  
    ID_Prodotto INT,  
    FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente),  
    FOREIGN KEY (ID_Prodotto) REFERENCES Prodotto(ID_Prodotto)  
);
```

Definizione di Weak Entity (Entità Debole).

Una Weak Entity (Entità Debole) è un'entità che non possiede un attributo chiave primaria sufficiente per identificarla in modo univoco e che, per esistere, dipende da un'altra entità, chiamata Strong Entity (Entità Forte).

Possiede un'identificatore parziale (un sottoinsieme di attributi che, da solo, non è sufficiente per l'identificazione).

L'identificatore completo si ottiene combinando la chiave primaria dell'entità forte con l'identificatore parziale dell'entità debole.



Ufficio è un'entità debole.

```

CREATE TABLE Dipartimento (
    ID_Dipartimento INT PRIMARY KEY,
    Nome VARCHAR(100)
);

CREATE TABLE Ufficio (
    ID_Dipartimento INT,
    Numero_Ufficio INT,
    Piano INT,
    PRIMARY KEY (ID_Dipartimento, Numero_Ufficio), -- PK composta
    FOREIGN KEY (ID_Dipartimento) REFERENCES
    Dipartimento(ID_Dipartimento) -- Relazione identificante
);

Dipartimento(Id_dipartimento, nome);
Ufficio(Numero_ufficio, ID_Dipartimento, Piano);
  
```

La chiave primaria di una Weak Entity è composta da la PK della Strong Entity + un identificatore parziale. Si usano quando un'entità ha senso di esistere solo nel contesto di un'altra.

A volte, per ovviare alla mancanza di una chiave primaria e per evitare la presenza di entità deboli, viene aggiunta una **chiave artificiale**.

Se utilizziamo una **chiave artificiale (Artificial Key)**, allora la chiave primaria dell'entità **Ufficio** non deve più essere composta da **(ID_Dipartimento, Numero_Ufficio)**, ma può essere rappresentata da un nuovo identificatore univoco, come **ID_Ufficio**.

In questo caso:

- **ID_Ufficio** diventa la **PRIMARY KEY** della tabella **Ufficio**.
- **ID_Dipartimento** rimane una **FOREIGN KEY**, ma non è più parte della chiave primaria

```
CREATE TABLE Dipartimento (
```

```
    ID_Dipartimento INT PRIMARY KEY,
```

```
    Nome VARCHAR(100)
```

```
);
```

```
CREATE TABLE Ufficio (
```

```
    ID_Ufficio INT PRIMARY KEY AUTO_INCREMENT, -- Chiave  
artificiale
```

```
    ID_Dipartimento INT, -- Chiave esterna, ma non più PK
```

```
    Numero_Ufficio INT,
```

```
    Piano INT,
```

```
    FOREIGN KEY (ID_Dipartimento) REFERENCES  
Dipartimento(ID_Dipartimento)
```

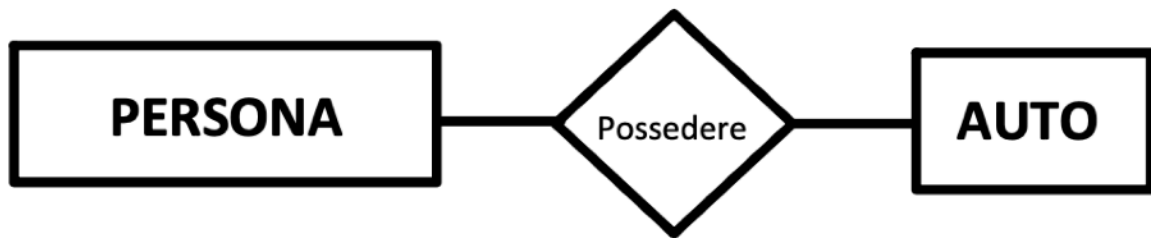
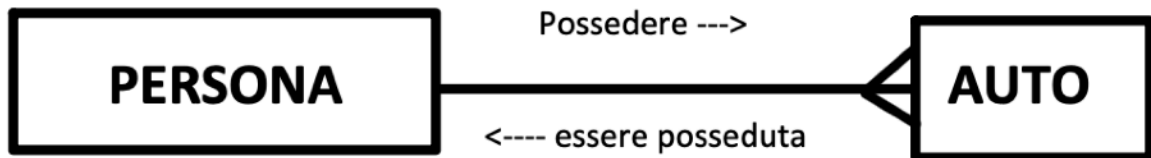
```
);
```

```
Dipartimento(Id_dipartimento, nome);
```

```
Ufficio(ID_Ufficio, ID_Dipartimento, Piano);
```

RELAZIONI

Ogni relazione è rappresentata graficamente da una linea che connette due entità da associare.



La molteplicità di una relazione è il numero di possibili istanze di un'entità messe in corrispondenza con un'istanza dell'altra entità che partecipa alla relazione.

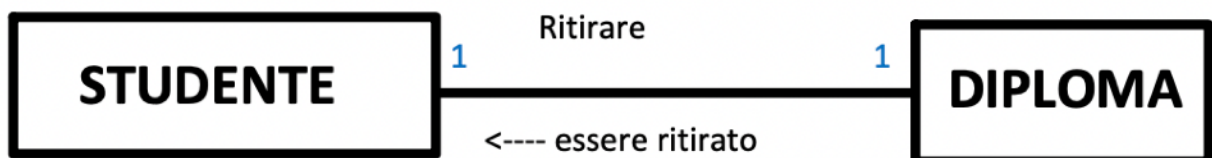
Le relazioni seguono le seguenti cardinalità:

1 : 1

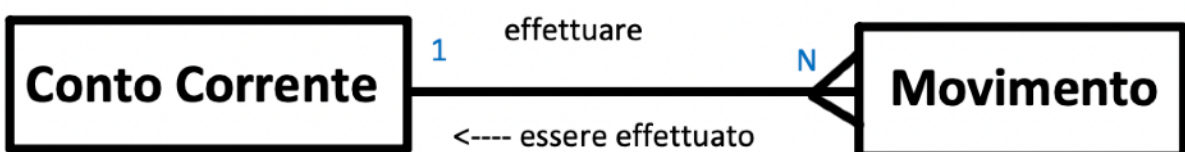
1 : N

N : N

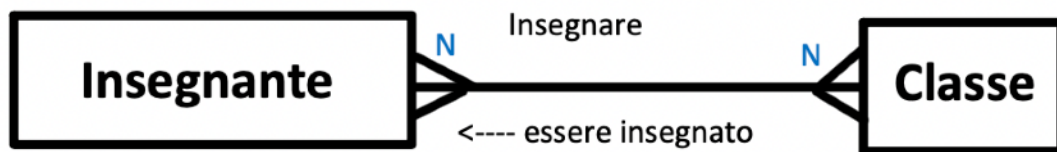
Esempi:



Ogni studente può ritirare un solo diploma.
Ogni diploma può essere ritirato da un solo studente.



Ogni conto corrente può effettuare più movimenti.
Ogni movimento è effettuato da un solo conto corrente.



Ogni insegnante può insegnare in una o più classi.
In ogni classe può insegnarci più di un insegnante.

DAL MODELLO CONCETTUALE AL MODELLO LOGICO.

Regole di derivazione.

- Ogni entità diventa una tabella;
- Ogni attributo diventa il nome di una colonna;
- La relazione 1:1 diventa un'unica tabella contenente gli attributi delle due entità.
- La relazione 1:N permette alla chiave primaria della relazione con cardinalità 1 di diventare chiave esterna della relazione con cardinalità N.
- Una relazione **N:N** (multi-a-molti) viene tradotta introducendo una **tabella di associazione** (o tabella intermedia). Questa tabella contiene almeno due **foreign key**, ciascuna riferita alle PK coinvolte nella relazione, e può avere una **chiave primaria composta** dalle due foreign key.

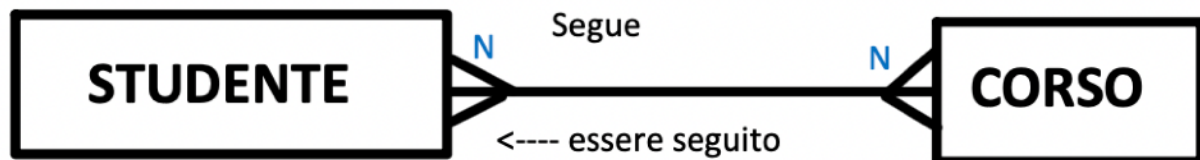
La chiave primaria composta la uso quando ciascun elemento dell'entità A non si può ripetere con un elemento dell'entità B, o viceversa. Se la relazione può avere **duplicati validi**, si può usare una chiave primaria autonoma nella tabella intermedia e mantenere le foreign key con un vincolo di unicità. Nella tabella di associazione che rappresenta una relazione **N:N**, le **foreign key (FK)** che fanno riferimento alle due tabelle

principali spesso **costituiscono la chiave primaria (PK) composta**.

- Se la relazione N:N non permette duplicati, le FK sono **anche** la PK (approccio più comune).
- Se servono identificatori separati o si vogliono registrare più occorrenze della stessa coppia FK, allora la PK è autonoma e le FK hanno solo un vincolo di unicità.

PROGETTIAMO I DATABASE.

Esempio:



Uno studente può seguire uno o più corsi.

Un corso può essere seguito da uno o più studenti.

Questo è il modello concettuale.

Applichiamo le regola di derivazione e troviamo il modello logico:

```
Studente(Id_studente, nome, cognome).  
Iscrizione(Id_studente, Id_Corso, data_iscrizione).  
Corso(Id_corso, nome_corso).
```

Se ora definisco nella tabella intermedia le due chiavi esterne `Id_studente` e `Id_corso` come chiavi primarie diventa una chiave composta, e **la chiave primaria composta (`id_studente`, `id_corso`) garantisce che uno studente non possa iscriversi due volte allo stesso corso.**

<u>id_studente</u>	<u>id_corso</u>	data_iscrizione
1	10	2021-10-09
2	8	2023-09-12
1	6	1900-08-09
(ERRORE)1	(ERRORE)10	//

Se la relazione può avere duplicati validi, si può usare una chiave primaria autonoma (id_iscrizione) e mantenere le foreign key.

```
CREATE TABLE Studente
(
id_studente INT PRIMARY KEY,
nome VARCHAR(50),
cognome VARCHAR(50)
);
```

```
CREATE TABLE Corso
( id_corso INT PRIMARY KEY,
  nome_corso VARCHAR(100)
);
```

```
CREATE TABLE Iscrizione
( id_studente INT,
  id_corso INT,
  data_iscrizione DATE,
  PRIMARY KEY (id_studente, id_corso),
  FOREIGN KEY (id_studente) REFERENCES Studente(id_studente),
  FOREIGN KEY (id_corso) REFERENCES Corso(id_corso)
);
```

E se voglio memorizzare più iscrizioni dello stesso studente allo stesso corso in momenti diversi? Chiave primaria a sé.

```
CREATE TABLE Iscrizione (
    id_iscrizione SERIAL PRIMARY KEY, -- PK autonumerico
    id_studente INT,
    id_corso INT,
    data_iscrizione DATE,
    FOREIGN KEY (id_studente) REFERENCES
Studente(id_studente),
    FOREIGN KEY (id_corso) REFERENCES Corso(id_corso),
    UNIQUE (id_studente, id_corso, data_iscrizione) --
Permette più iscrizioni, ma in date diverse
);
```

Permette di registrare più iscrizioni dello stesso studente a un corso in momenti diversi.

Se UNIQUE è applicato a più colonne, garantisce che **la combinazione** di quei valori sia univoca.

Teorema: Due foreign key possono fare riferimento alla stessa primary key, ma con ruoli distinti.

Esercizio 1 Progettazione .

Una compagnia aerea offre voli che possiedono un numero che identifica la tratta (per esempio, Roma-Milano), una data (25 marzo 2001), un orario di partenza (ore 08:00), uno di arrivo (ore 09:00), un aeroporto di partenza e uno di destinazione.

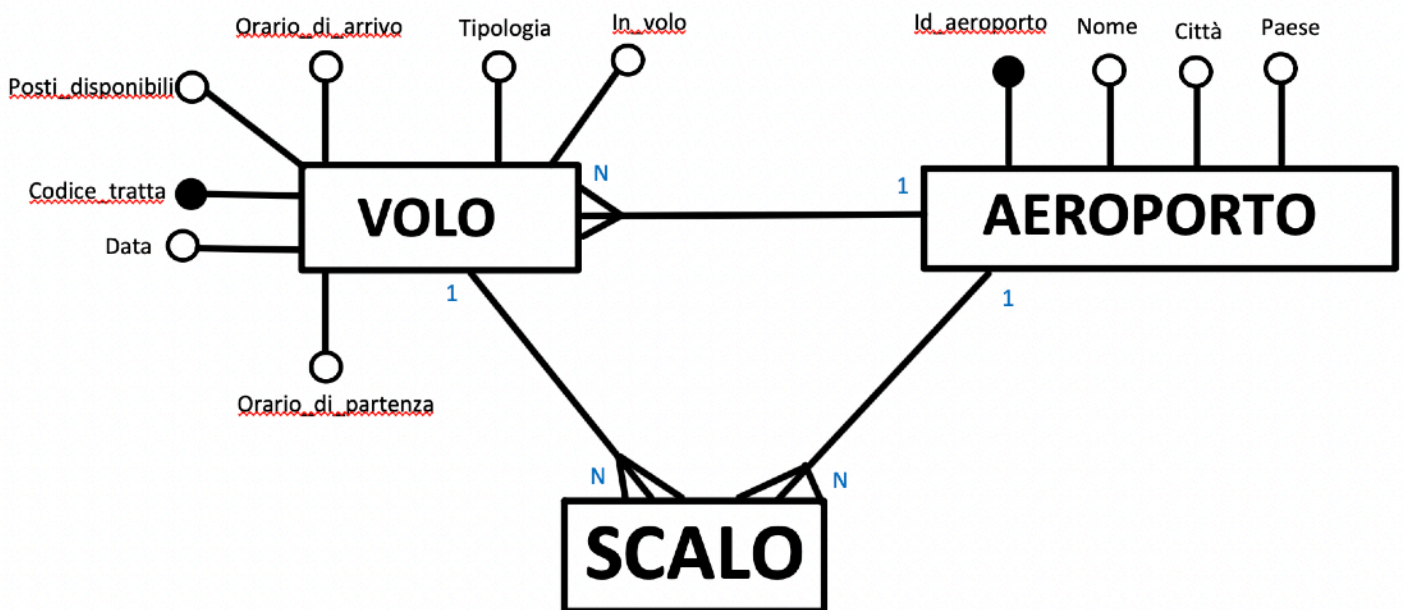
Ci sono voli nazionali e internazionali.

I voli internazionali possono avere uno o più scali.

Dei voli passati è di interesse l'orario reale di partenza e di arrivo (per esempio, con riferimento al volo suddetto, ore 08:05 e 09:07), di quelli futuri è di interesse il numero di posti disponibili.

Progettare schema ER/modello logico e modello fisico.

Soluzione.



Questo è lo schema E/R, pertanto il modello logico sarà il seguente:

Volo(Codice_Tratta, Data, Orario_di_partenza, Orario_di_arrivo, tipologia, in_volo, posti_disponibili, aeroporto_di_partenza, aeroporto_di_arrivo).

Aeroporto(id_aeroporto, Nome, città, paese).

Scalo(Codice tratta, Id_aeroporto).

Si potrebbe aggiungere una tabella **Storico_Voli** per memorizzare gli orari reali di partenza e arrivo dei voli passati. Comunque è un primo schizzo abbozzato.

Ma vediamo come sono state create le tabelle in linguaggio SQL:

```
CREATE TABLE Volo (  
    Codice_Tratta INT PRIMARY KEY,  
    data DATE NOT NULL,  
    orario_di_partenza TIME,  
    orario_di_arrivo TIME,  
    Tipologia ENUM('Nazionale', 'Internazionale') NOT NULL,  
    in_volo ENUM('SI', 'NO') NOT NULL,  
    posti_disponibili INT CHECK (posti_disponibili >= 0),  
    -- Solo per voli futuri, Significa che la colonna posti_disponibili non può mai  
    -- avere valori negativi. Se qualcuno prova a inserire un valore minore di 0, il database  
    -- genera un errore.  
    aeroporto_di_partenza VARCHAR(50) NOT NULL,  
    aeroporto_di_arrivo VARCHAR(50) NOT NULL,  
    FOREIGN KEY (aeroporto_di_partenza) REFERENCES  
Aeroporto(id_aeroporto) ON DELETE CASCADE,  
    FOREIGN KEY (aeroporto_di_arrivo) REFERENCES  
Aeroporto(id_aeroporto) ON DELETE CASCADE  
  
    Se un aeroporto viene cancellato, tutti i voli associati vengono eliminati  
    automaticamente.  
);
```

```
CREATE TABLE Scalo (
```

```
    Codice_Tratta INT NOT NULL,
```

```
    Id_aeroporto VARCHAR(50) NOT NULL,
```

```
    PRIMARY KEY (Codice_Tratta, Id_aeroporto),
```

```
    FOREIGN KEY (Codice_Tratta) REFERENCES
```

```
Volo(Codice_Tratta) ON DELETE CASCADE,
```

Se un volo viene cancellato dalla tabella Volo, tutti i suoi scali nella tabella Scalo devono essere eliminati automaticamente.

```
    FOREIGN KEY (Id_aeroporto) REFERENCES
```

```
Aeroporto(id_aeroporto) ON DELETE CASCADE
```

Se un aeroporto viene eliminato, tutti gli scali in quell'aeroporto vengono eliminati automaticamente.

```
);
```

```
CREATE TABLE Aeroporto (
```

```
    id_aeroporto VARCHAR(50) PRIMARY KEY,
```

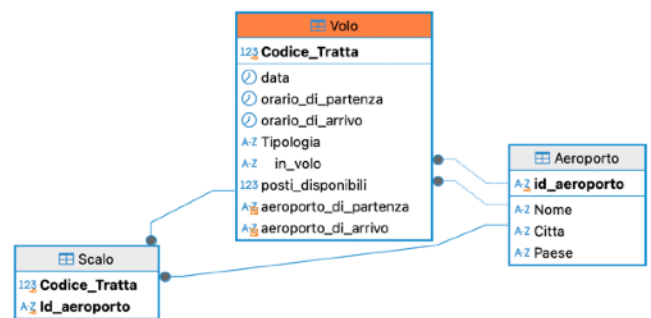
```
    Nome VARCHAR(100) NOT NULL,
```

```
    Città VARCHAR(50) NOT NULL,
```

```
    Paese VARCHAR(50) NOT NULL
```

```
);
```

Quando si applica **ON DELETE CASCADE** a una **foreign key**, se il record nella tabella **padre** viene eliminato, allora **l'intera tupla (riga) della tabella figlia viene eliminata**. Non vengono cancellati solo i valori dei campi, ma **l'intero record nella tabella figlia viene rimosso!**



Ora implementiamo il database a livello fisico:

Le tabelle sono le seguenti:

Aeroporto								
id_aeroporto		Nome			Citta		Paese	
AMS		Amsterdam Schiphol			Amsterdam		Paesi Bassi	
ATL		Atlanta			Atlanta		USA	
BKK		Bangkok			Bangkok		Thailandia	
CDG		Charles de Gaulle			Parigi		Francia	
DXB		Dubai			Dubai		Emirati Arabi Uniti	
FCO		Fiumicino			Roma		Italia	
HND		Tokyo Haneda			Tokyo		Giappone	
ICN		Incheon			Seoul		Corea del Sud	
JFK		John F. Kennedy			New York		USA	
LAX		Los Angeles			Los Angeles		USA	
LHR		Heathrow			Londra		Regno Unito	
ORD		Chicago O'Hare			Chicago		USA	
PEK		Beijing Capital			Pechino		Cina	
SIN		Singapore Changi			Singapore		Singapore	
SYD		Sydney			Sydney		Australia	

Volo								
Codice_Tratta	data	orario_di_partenza	orario_di_arrivo	Tipologia	in_volo	posti_disponibili	aeroporto_di_partenza	aeroporto_di_arrivo
3.001	2024-03-01			Internazionale	NO	500	FCO	JFK
3.002	2024-03-02			Nazionale	NO	1.000	LHR	SYD
3.003	2024-03-03			Internazionale	NO	750	CDG	DXB
3.004	2024-03-04			Internazionale	NO	1.200	JFK	FCO
3.005	2024-03-05			Nazionale	NO	900	SYD	LHR
3.006	2024-03-06			Internazionale	NO	400	DXB	CDG
3.007	2024-03-07			Nazionale	NO	1.300	FCO	JFK
3.008	2024-03-08			Internazionale	NO	1.100	LHR	SYD
3.009	2024-03-09			Nazionale	NO	950	CDG	DXB
3.010	2024-03-10			Internazionale	NO	600	JFK	FCO
3.011	2024-03-11	08:00:00	12:00:00	Internazionale	SI		SYD	LHR
3.012	2024-03-12	12:30:00	16:45:00	Nazionale	SI		DXB	CDG
3.013	2024-03-13	14:00:00	18:30:00	Internazionale	SI		FCO	JFK
3.014	2024-03-14	09:45:00	14:10:00	Nazionale	SI		LHR	SYD
3.015	2024-03-15	16:00:00	19:50:00	Internazionale	SI		CDG	DXB
3.016	2024-03-16	07:30:00	10:15:00	Internazionale	SI		JFK	FCO
3.017	2024-03-17	15:15:00	18:30:00	Nazionale	SI		SYD	LHR
3.018	2024-03-18	11:00:00	14:20:00	Internazionale	SI		DXB	CDG
3.019	2024-03-19	18:00:00	22:30:00	Nazionale	SI		FCO	JFK
3.020	2024-03-20	10:00:00	14:15:00	Internazionale	SI		LHR	SYD

Scale	
Codice_Tratta	Id_aeroporto
3.013	ATL
3.020	BKK
3.004	CDG
3.016	CDG
3.003	HND
3.006	HND
3.004	LAX
3.008	LAX
3.001	LHR
3.006	LHR
3.010	LHR
3.018	LHR
3.011	PEK
3.003	SIN
3.013	SIN
3.020	SIN
3.015	SYD

Facciamo 3 query per cominciare a prendere confidenza:

Query 1.

Trovare la lista degli aeroporti.

```
SELECT Nome FROM Aeroporto;
```

Query 2.

Trovare tutti i voli in arrivo a Fiumicino.

```
SELECT Codice_Tratta FROM Volo WHERE aeroporto_di_arrivo ="FCO";
```

Query 3.

Trovare tutti i voli internazionali passati, partiti dopo le 9:00, che hanno effettuato almeno uno scalo a Singapore.

```
SELECT Volo.Codice_Tratta  
      FROM Volo,Scalo  
WHERE Volo.Codice_Tratta = Scalo.Codice_Tratta  
      and Volo.`in_volo` ="SI"  
      and Volo.orario_di_partenza > '09:00:00'  
      and Scalo.Id_aeroporto ="SIN";
```

Output: 3013,3020.

Al momento con le query mi fermo qui. Fidatevi che ci sarà tanto da dire, ora impariamo a creare i modelli concettuali.

:)

Esercizio 2 - Progettazione schema E/R con notazione Chen.

Una compagnia aerea offre voli che possiedono un numero che identifica la tratta (per esempio, Roma-Milano), una data (25 marzo 2001), un orario di partenza (ore 08:00), uno di arrivo (ore 09:00), un aeroporto di partenza e uno di destinazione.

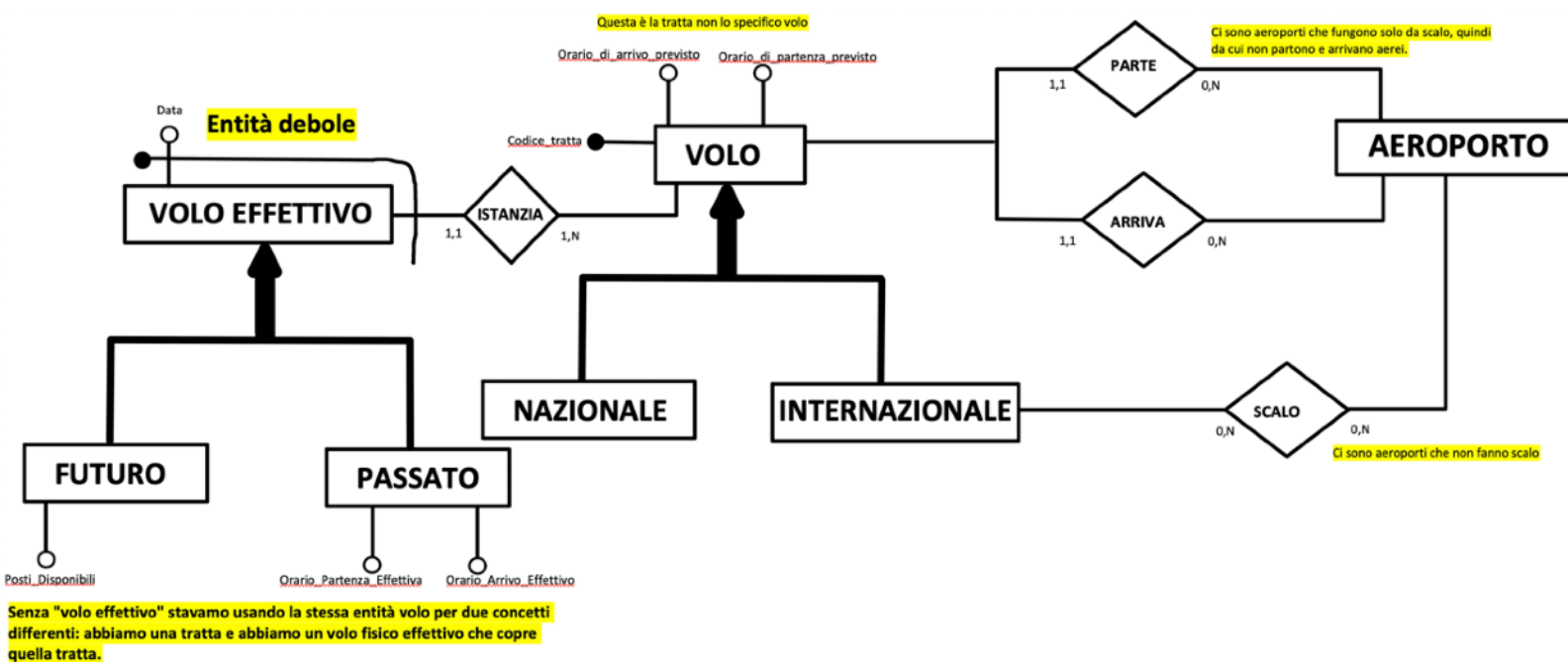
Ci sono voli nazionali e internazionali.

I voli internazionali possono avere uno o più scali.

Dei voli passati è di interesse l'orario reale di partenza e di arrivo (per esempio, con riferimento al volo suddetto, ore 08:05 e 09:07), di quelli futuri è di interesse il numero di posti disponibili.

Progettare schema ER.

Soluzione.



Esercizio 3 - Relazione ternaria .

Si consideri una relazione ternaria che coinvolge le seguenti entità: **impiegato**, **progetto** e **consulente**.

Indicare in quali dei seguenti casi (e, in caso affermativo, come) è opportuno sostituire a tale relazione due (o tre) relazioni binarie.

Ogni impiegato è coinvolto in zero o più progetti e interagisce con zero o più consulenti.

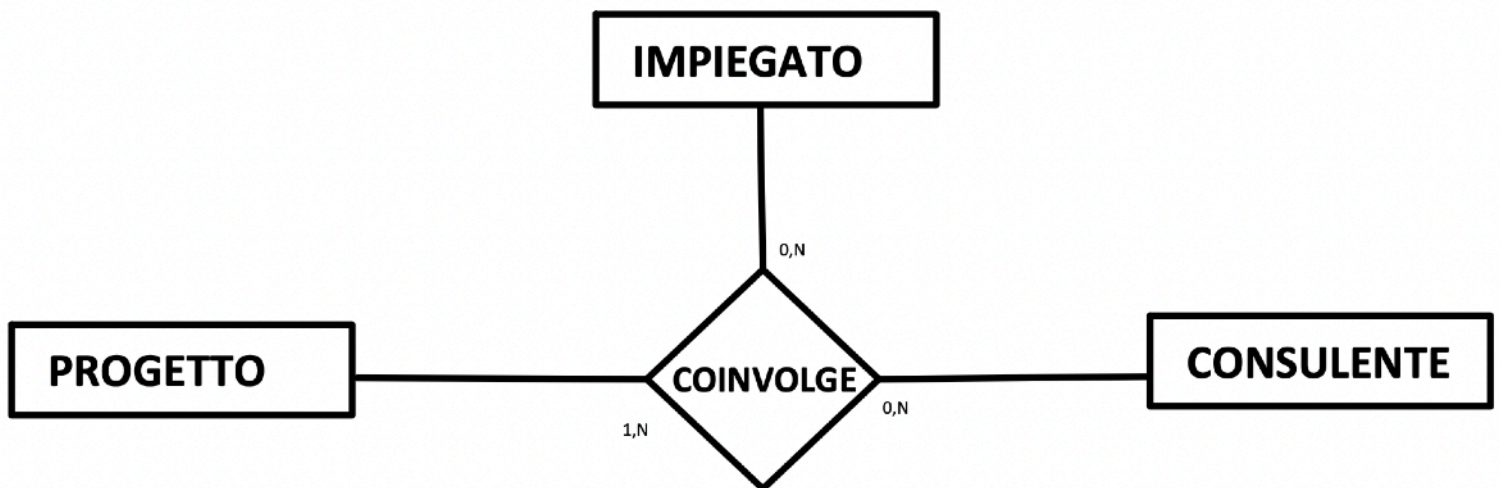
Ogni consulente è coinvolto in zero o più progetti e interagisce con zero o più impiegati.

Ogni progetto coinvolge uno o più impiegati e uno o più consulenti (che possono non interagire tra di loro).

Un impiegato e un consulente collaborano nell'ambito di un progetto se e solo se essi collaborano fra loro e sono entrambi coinvolti nello stesso progetto.

Soluzione.

La relazione ternaria non cattura tutta la specifica.

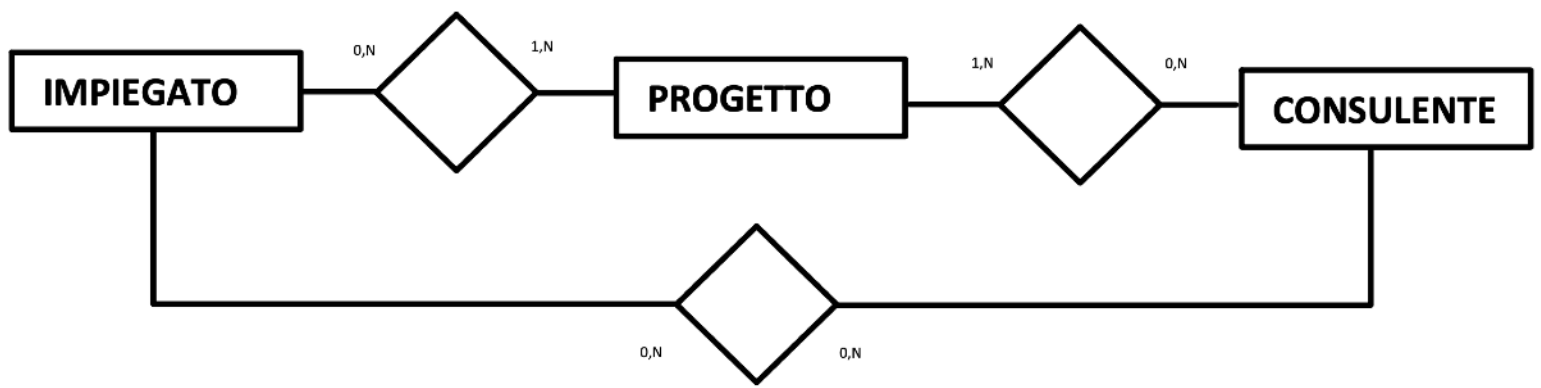


Utilizzando le cardinalità che mi vengono suggerite dal testo non sono sicuro di poter catturare situazioni in cui, per esempio, un progetto coinvolge 100 consulenti e nessun impiegato oppure un consulente che lavora su 10 progetti senza impiegati.

Queste cardinalità, con la relazione ternaria, rischiano di catturare situazioni che non sono rappresentate dalla specifica.

In casi di questo tipo, il modo migliore per rendere "non ambiguo" questo oggetto è **esplodere la relazione**.

Anziché mettere in relazione tra di loro le tre entità, espando man mano le relazioni a seconda di quello che mi viene richiesto:



Un impiegato lavora da 0 a N progetti, in un progetto ci deve essere almeno un impiegato.

In un progetto ci deve essere almeno un consulente, un consulente può lavorare da 0 a N progetti.

I consulenti collaborano con 0 fino ad N impiegati e viceversa.

In questo modo, cerco di rendere esplicito il più possibile tutto quello che viene richiesto dalla specifica.

Esercizio 4 - Progettazione schema E/R con notazione Chen.

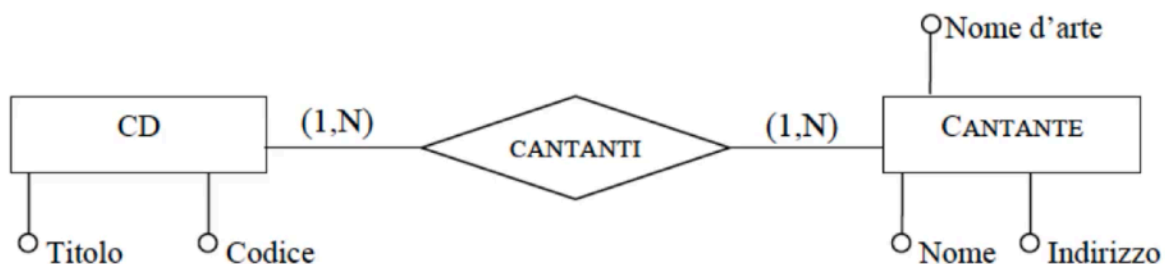
Una casa discografica produce dischi aventi un codice ed un titolo; ogni disco è inciso da uno o più cantanti, ognuno dei quali ha un nome, un indirizzo e, **qualcuno**, un nome d'arte.

Progettare schema ER.

Soluzione.

Prima nome d'arte facciamolo attributo di cantante.

Poteva anche essere un'entità a sé con la generalizzazione parziale sotto cantante creano un sottoinsieme.

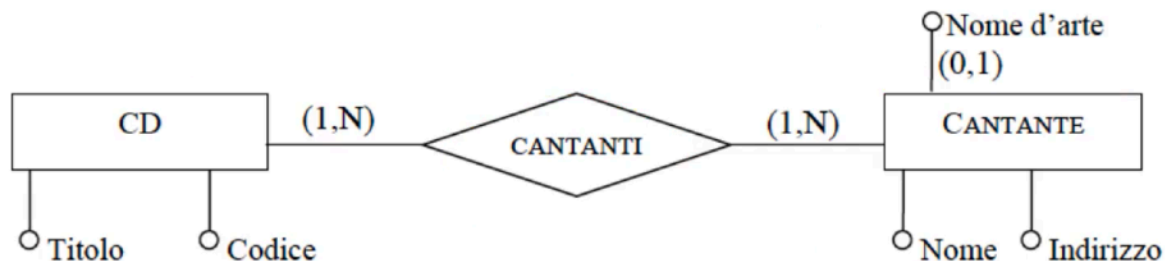


La relazione è CANTATI.

Per gestire questo nome d'arte possiamo usare la
CARDINALITÀ DELL'ATTRIBUTO.

Un cantante può possedere un nome d'arte oppure esserne privo, avendone quindi da un minimo di zero a un massimo di uno.

L'espressione "**qualcuno** (dei vari cantanti)" presente nel testo si riflette nella cardinalità dell'attributo, indicando che il nome d'arte è **opzionale** e può essere assegnato al più una volta per ciascun cantante.



Nel momento in cui l'attributo ha una cardinalità (0,1) parliamo di **attributo opzionale: può esserci o no**. Cantante diventa una tabella con nome e indirizzo NOT NULL, mentre nome d'arte potrebbe essere NULL in quanto non tutti lo possiedono.

Esercizio 5 - Progettazione schema E/R con notazione Chen.

Definire uno schema ER che descriva i dati di un'applicazione relativa all'**anagrafe** del comune di chissadove, con cittadini e famiglie.

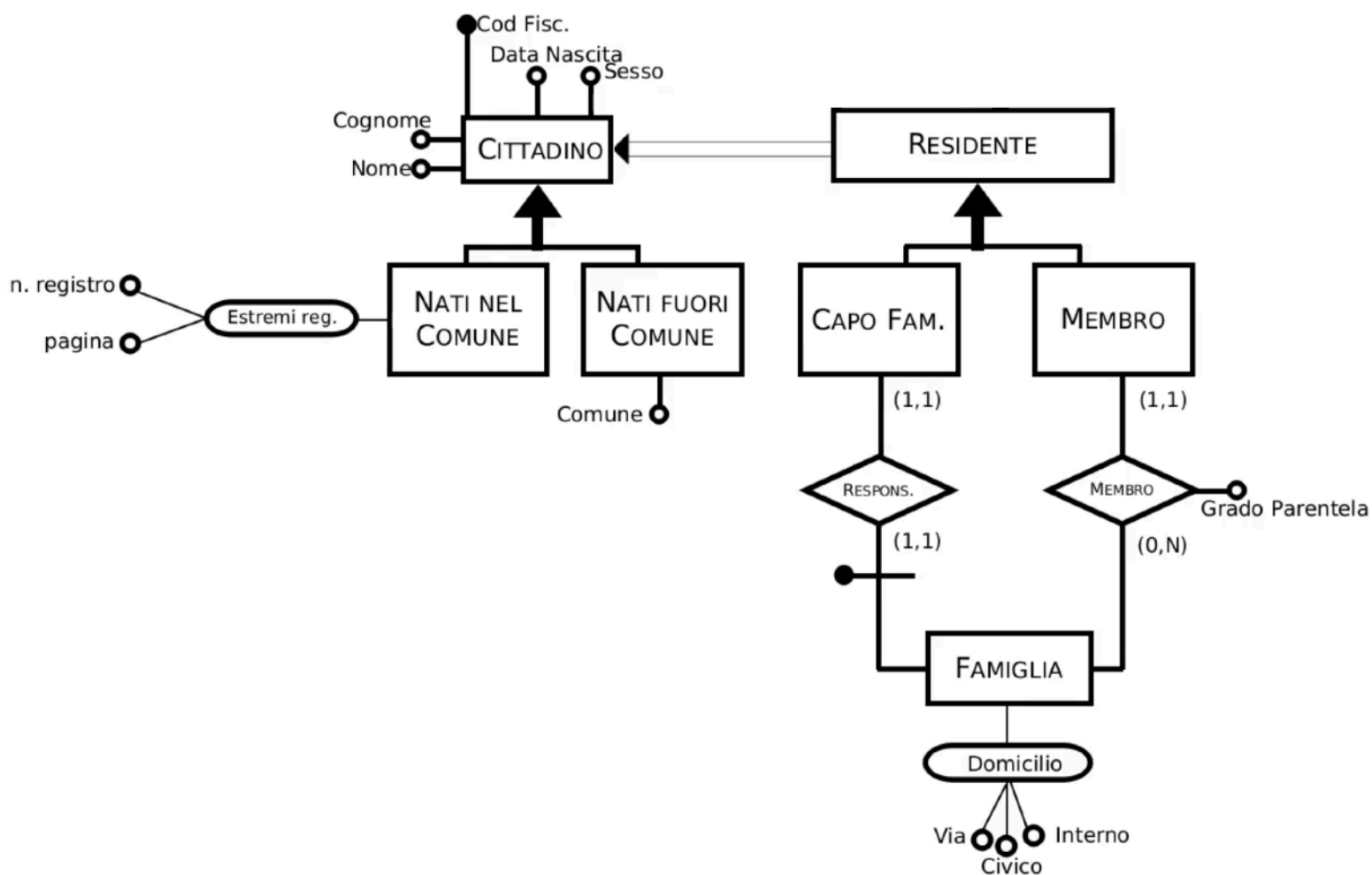
Vanno memorizzate:

- Informazioni sui cittadini nati nel comune e su quelli residenti in esso;
Ogni cittadino è identificato dal codice fiscale e ha cognome, nome, sesso e data di nascita;

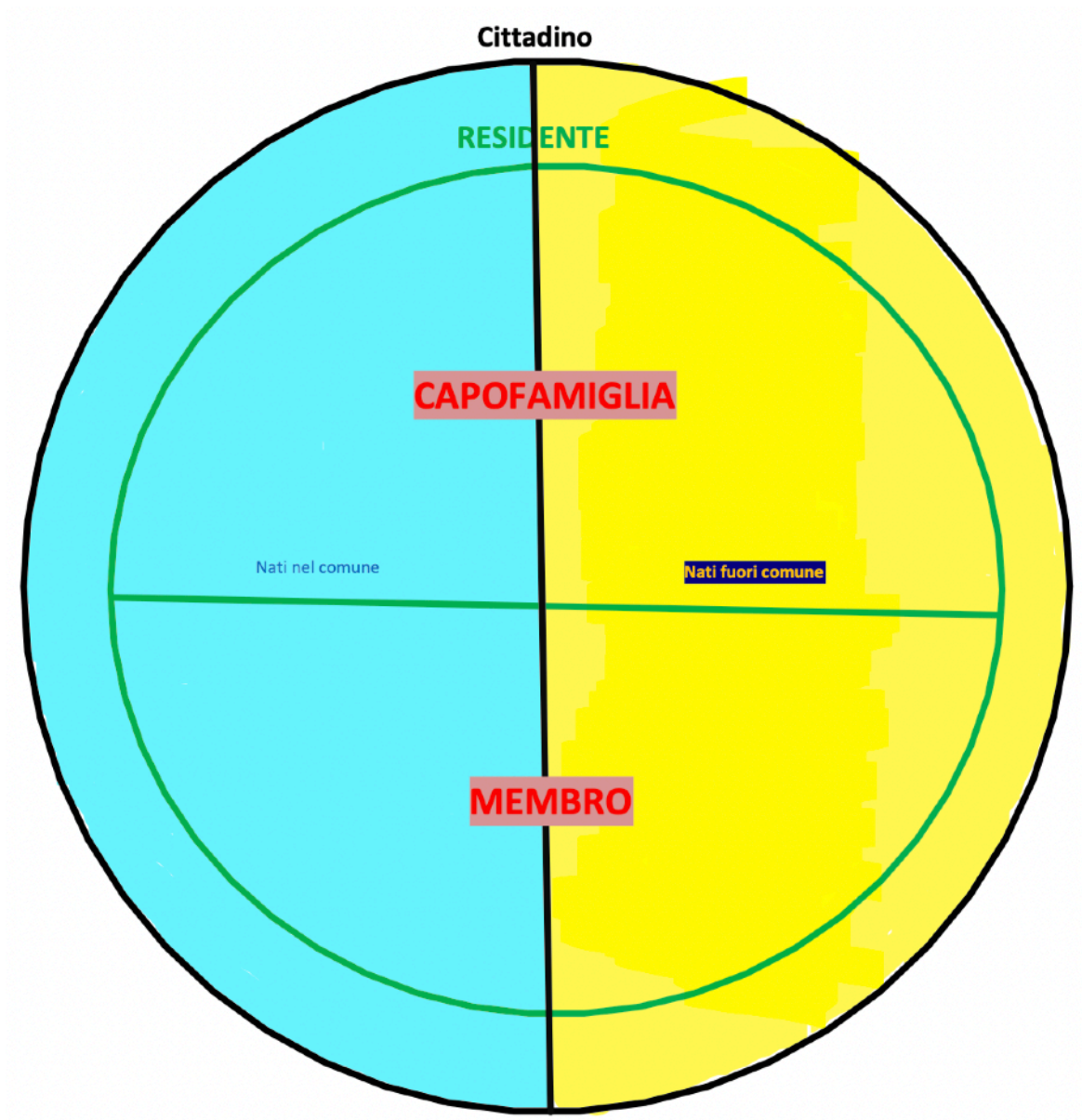
- Per i nati nel comune, sono registrati anche gli estremi di registrazione (numero del registro e pagina).
- Per i nati in altri comuni, è registrato il comune di nascita.
- Informazioni sulle famiglie residenti, ognuna delle quali ha uno e un solo capofamiglia e zero o più membri, per ognuno dei quali è indicato (con la sigla) il grado di parentela (coniuge, figlio, genitore o altro); ogni cittadino residente appartiene ad una e una sola famiglia; tutti i membri di una famiglia hanno lo stesso domicilio (via, numero civico, interno).

Progettare schema ER.

Soluzione.



Vado a differenziare, tramite una relazione di generalizzazione forte, tutti i cittadini nati nel comune e nati fuori comune, **alcuni cittadini sono residenti**. Alcuni residenti sono capi famiglia, altri residenti sono membri e questi ultimi partecipano alla famiglia con un grado di parentela, e la famiglia è identificata dal capofamiglia che è comunque un residente.



Ci sono membri di famiglie nati nel comune e membri di famiglie nati fuori comune.

Ci sono capo famiglia nati nel comune e capo famiglia nati fuori comune.

Un residente è per forza un cittadino.

ATTRIBUTO NELLE RELAZIONI: Se un attributo dipende direttamente dalla relazione e non da una singola entità, allora va associato alla relazione stessa.

È un'informazione che riguarda l'associazione.

Gli attributi devono appartenere alla relazione solo se dipendono direttamente da essa e non dalle entità collegate. Se l'attributo può essere spostato su un'entità senza perdita di informazioni, è meglio farlo.

Esercizio 6 - Progettazione schema E/R con notazione Chen.

Definire uno schema Entità-Relazione che descriva i dati di una applicazione relativa ad un **reparto ospedaliero**.

Sono di interesse le seguenti informazioni:

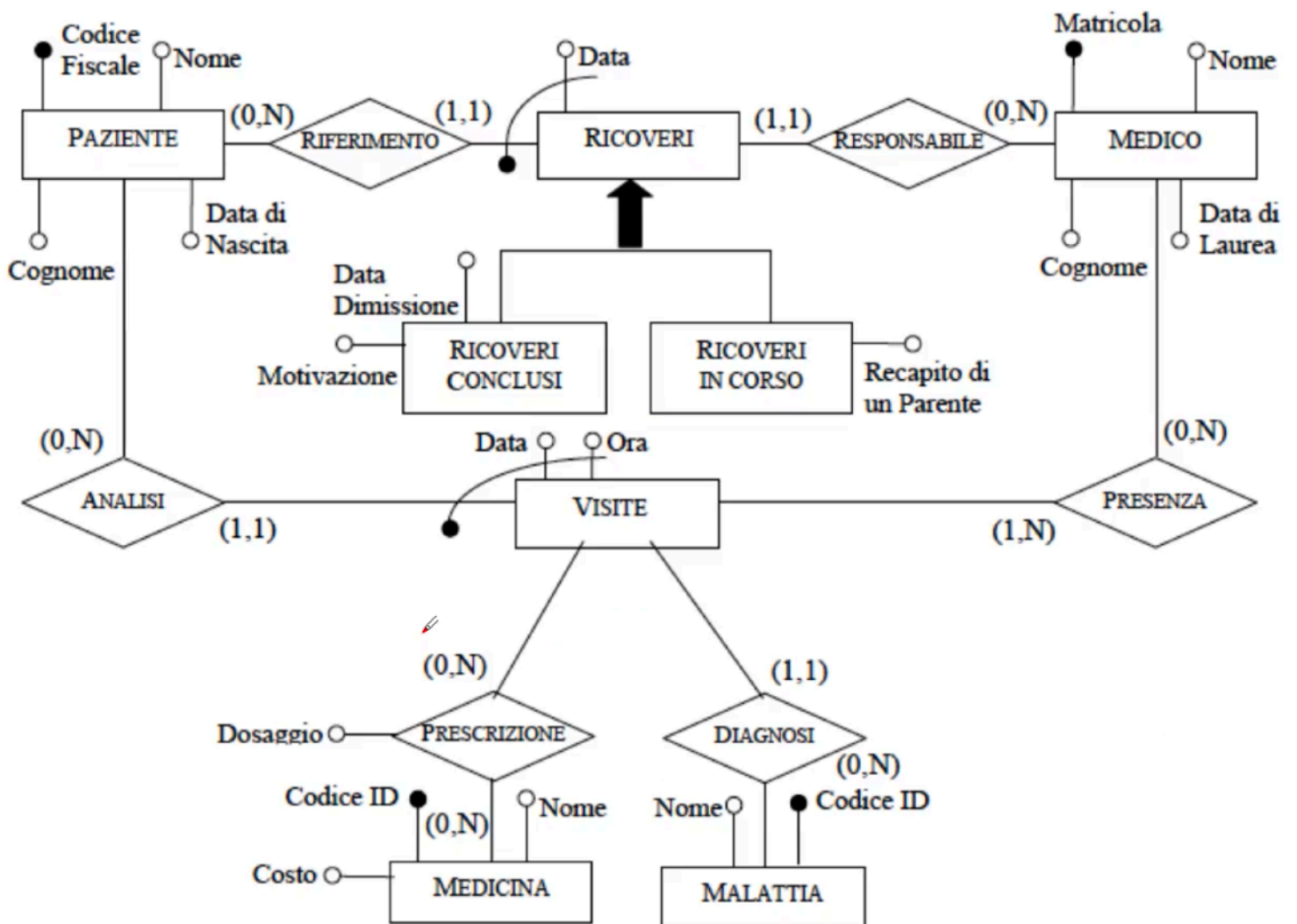
- I pazienti, con codice fiscale, nome, cognome e data di nascita.
- I ricoveri dei pazienti, ognuno con data di inizio (identificante nell'ambito dei ricoveri di ciascun paziente) e medico curante; inoltre, per i ricoveri conclusi, la data di conclusione e la motivazione (dimissione, trasferimento, etc.), e, per i ricoveri in corso, il recapito di un parente (che si può assumere sia semplicemente una stringa).
- I medici, con numero di matricola, cognome, nome e data di laurea.
- Le visite, con la data, l'ora, i medici visitanti, le medicine prescritte (con le relative quantità) e le malattie diagnosticate; ogni visita è identificata dal paziente coinvolto, dalla data e dall'ora.
- Per ogni medicina sono rilevanti un codice identificativo, un nome e un costo.
- Per ogni malattia sono rilevanti un codice identificativo e un nome.

Progettare schema ER.

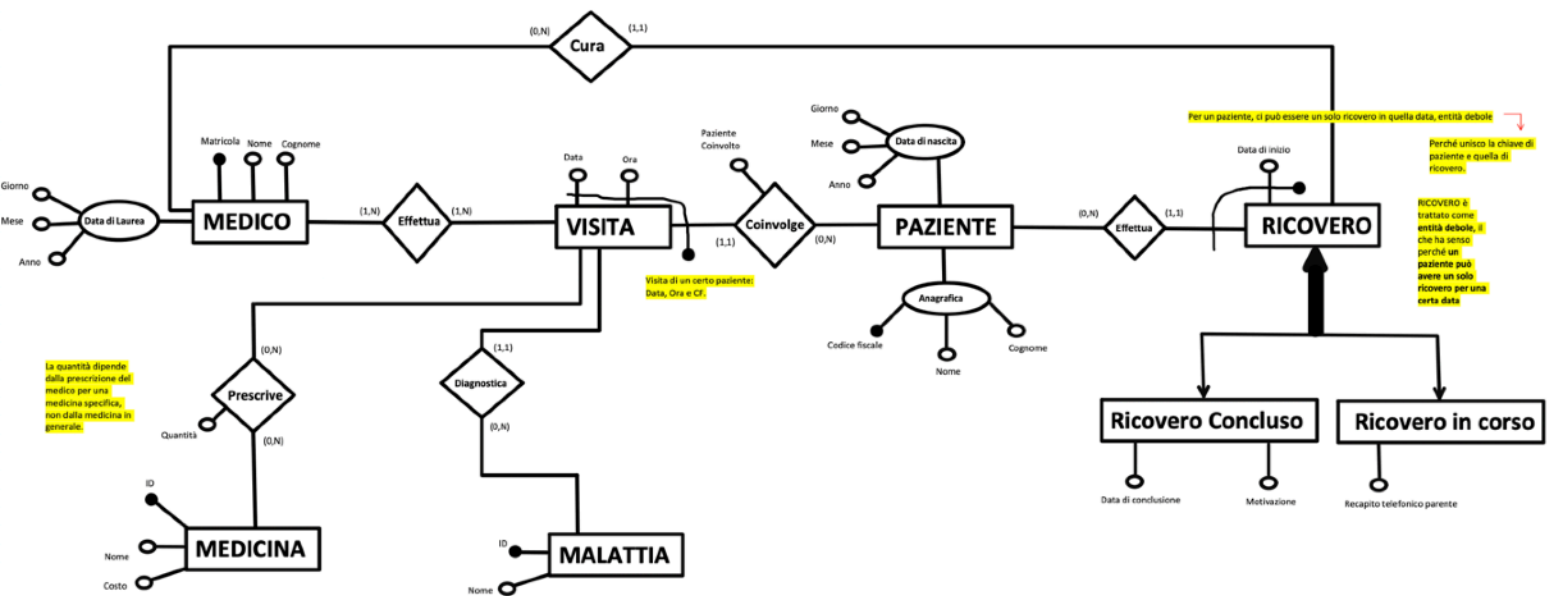
Soluzione.

Nota: Ogni entità forte può avere più entità deboli associate, ma ogni entità debole è legata a una sola entità forte. La chiave primaria dell'entità debole include la chiave primaria dell'entità forte.

Soluzione del Prof.Ing.A.Pellegrini:



Di seguito la mia soluzione che nessuno prenderà in considerazione :



Alla fine è identica.

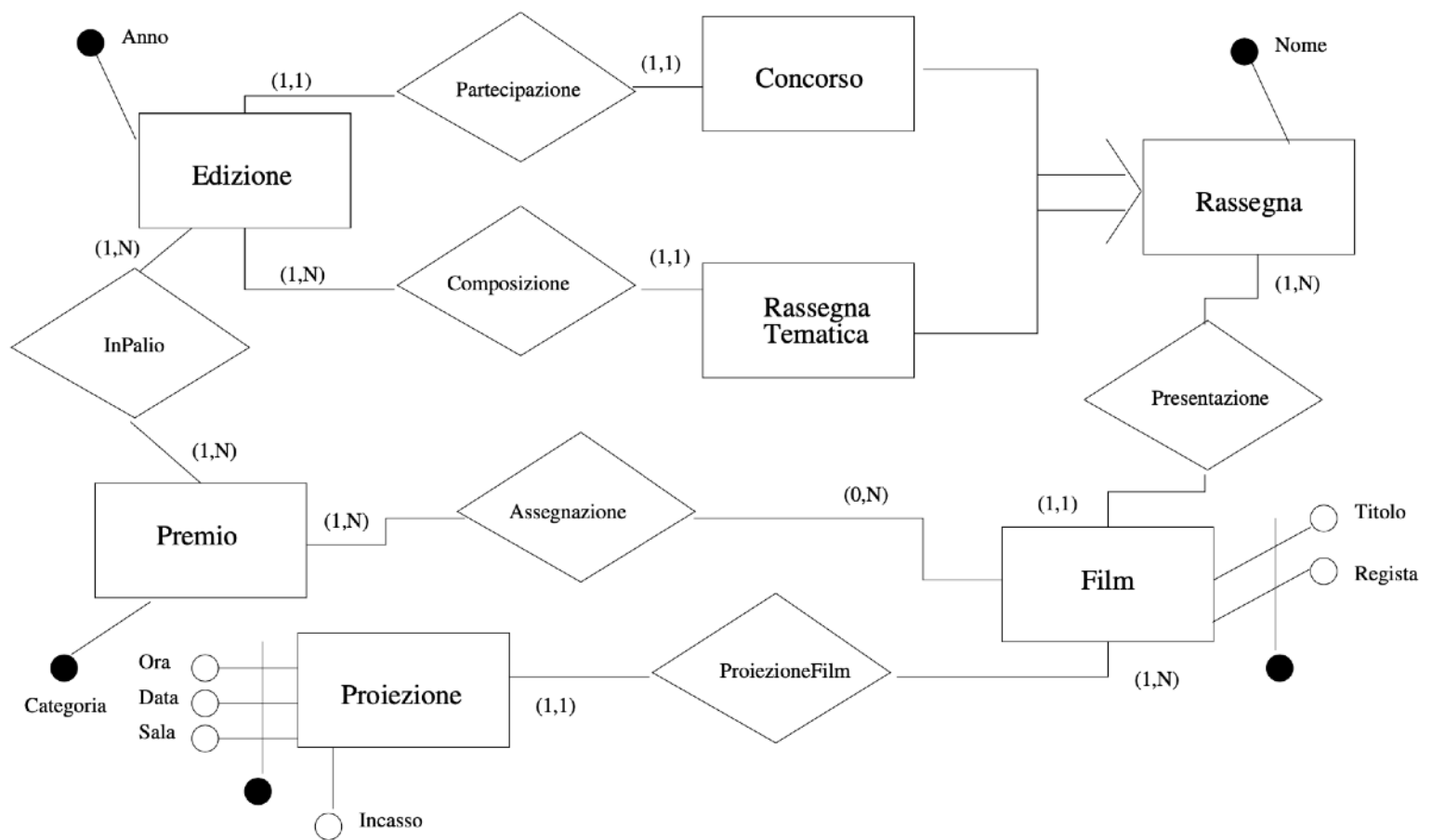
Esercizio 7 - Progettazione schema E/R con notazione Chen.

Mostrare uno schema ER che modelli la seguente realtà (scegliendo liberamente per gli aspetti lasciati indefiniti o ambigui nelle specifiche):

- Oggetto dell'interesse è una serie di **festival cinematografici** (più precisamente, si tratta delle varie edizioni, in anni diversi, dello stesso festival) ciascuno organizzato in **rassegne**, di cui una (per ciascun festival) destinata ai film in concorso e altre su temi diversi (identificate attraverso un nome); ogni film viene presentato in una sola rassegna di un solo festival;
- Per ogni film ci possono essere diverse proiezioni, di cui è importante indicare **sala, giorno, ora** e **incasso**;
- Per ogni festival, sono assegnati i seguenti premi: miglior film, miglior regia, miglior attore e miglior attrice (protagonista e non protagonista).

Progettare schema ER.

Soluzione.



Esercizio 8 - Progettazione schema E/R con notazione Chen.

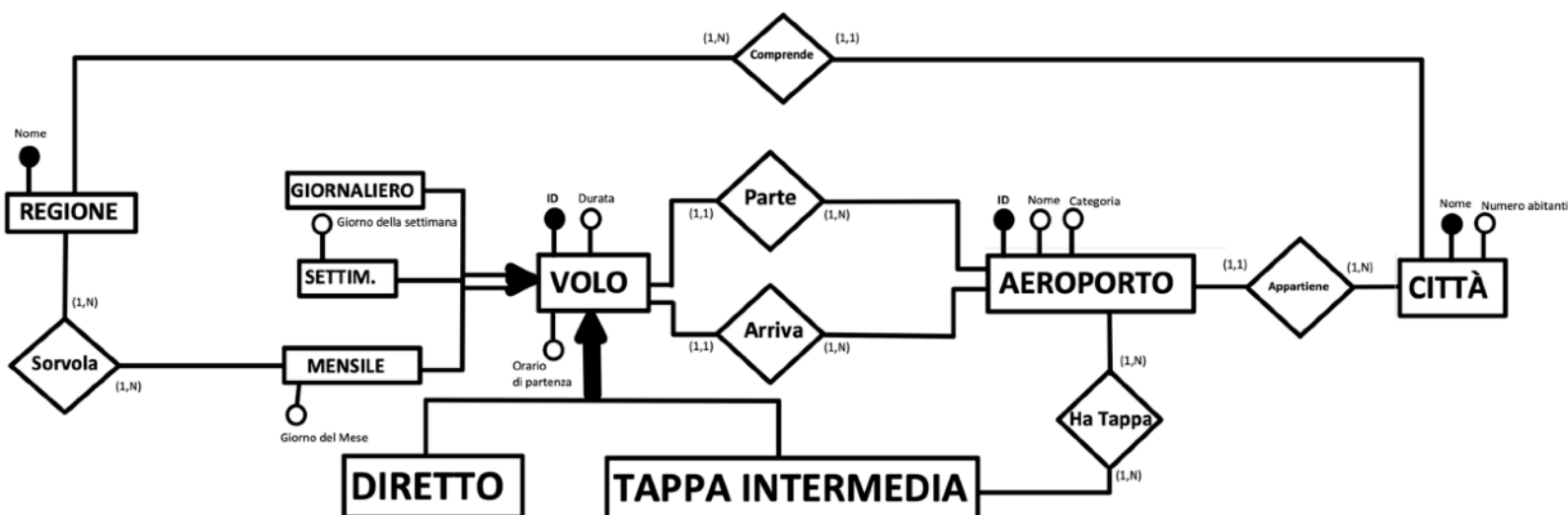
Si deve progettare la base di dati di una **compagnia aerea nazionale**, per la quale sono di interesse:

- I voli, con codice, durata in minuti, aeroporto di partenza e aeroporto di arrivo. Alcuni voli prevedono tappe intermedie in aeroporti diversi da quelli di partenza e arrivo, e delle tappe intermedie di un volo interessa l'ordine con cui esse si susseguono (ad esempio, il volo 124 da Milano Linate a Palermo Punta Raisi prevede prima l'aeroporto di Bologna e poi quello di Napoli come tappe intermedie). Ogni volo (sia esso diretto, oppure con tappe intermedie) appartiene ad uno ed uno solo dei seguenti tipi:
 - giornaliero (di questi voli interessa l'orario di partenza),
 - settimanali (di questi voli interessa il giorno della settimana e l'orario di partenza),
 - mensile (di questi voli interessa il giorno del mese, l'orario di partenza, e anche le regioni sorvolate).

- Gli aeroporti, con codice, nome, categoria e città (a sua volta con nome, numero di abitanti e regione).

Progettare schema ER.

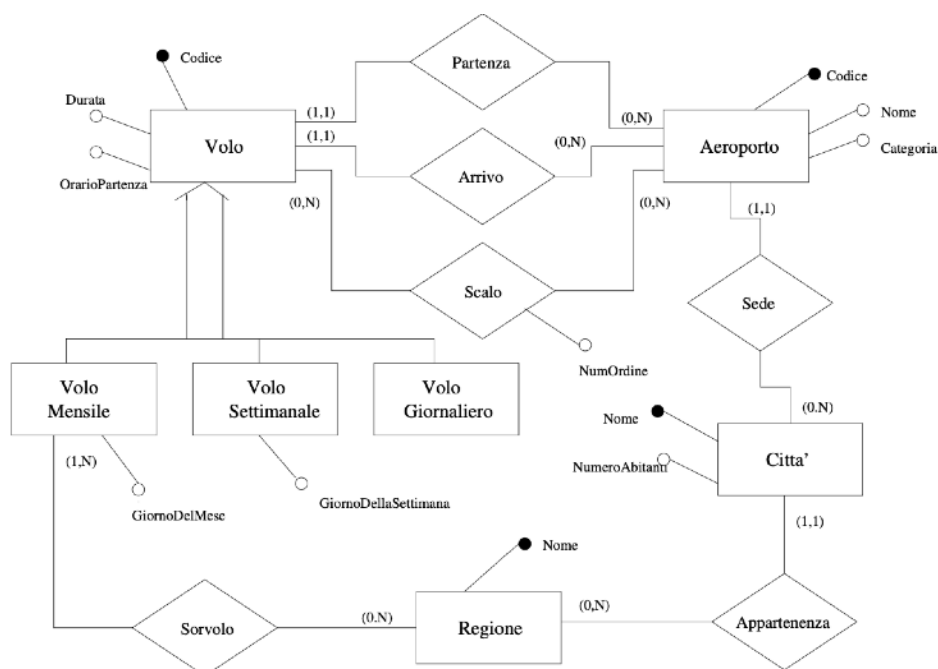
Soluzione.



Invece di collegare direttamente **VOLO** con gli aeroporti delle tappe intermedie, ho creato una relazione tra **Tappa Intermedia** e **Aeroporto**.

La soluzione sopra riportata è frutto della mia elaborazione, mentre di seguito presento quella proposta dal testo, che, a mio avviso, risulta errata in quanto non introduce una chiara distinzione tra voli diretti e voli con tappe intermedie.

Ritengo opportuno riportarla per completezza, sebbene in sede d'esame non avrei optato per un'impostazione concettualmente così imprecisa.



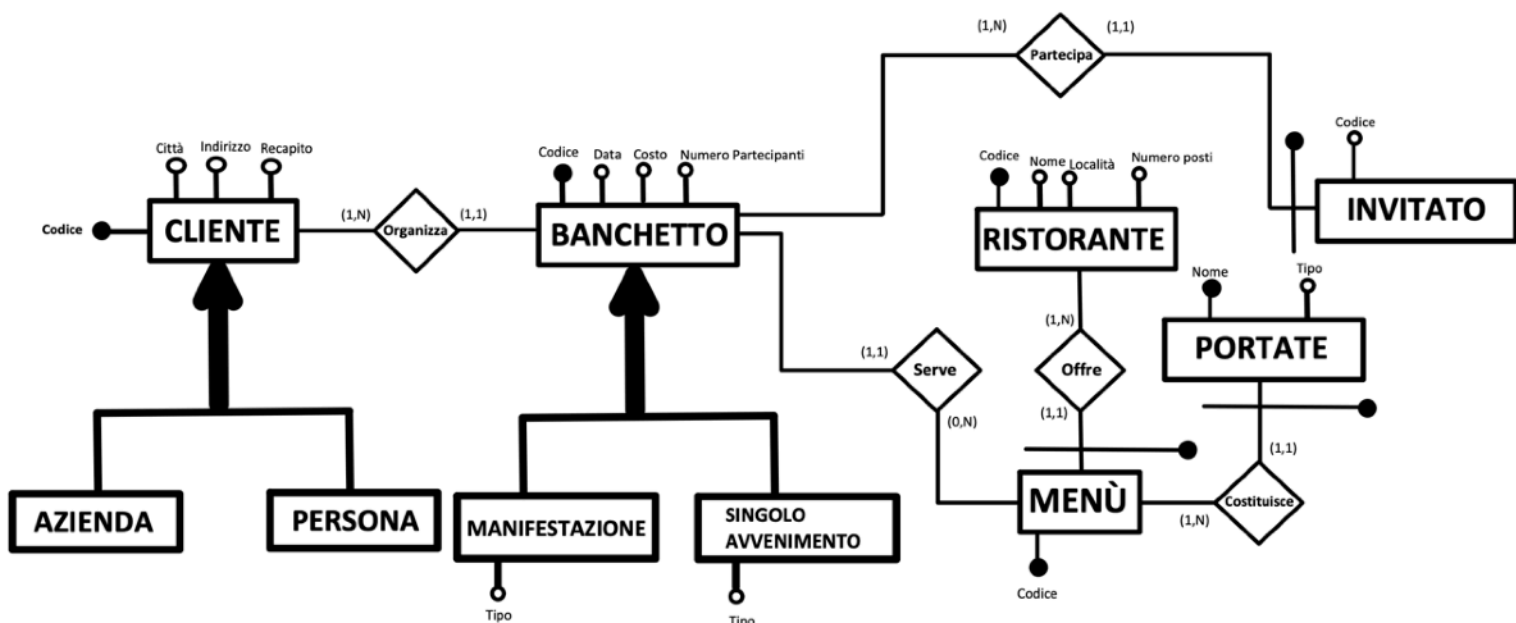
Esercizio 9 - Progettazione schema E/R con notazione Chen.

La base di dati di un'agenzia di pubbliche relazioni deve contenere le seguenti informazioni:

- un catalogo di clienti (codice fiscale, città, indirizzo, telefono), che possono essere aziende o persone fisiche;
- un insieme di informazioni su dei banchetti (codice, data, costo, numero partecipanti) organizzati nell'ambito di manifestazioni o in occasione di singoli avvenimenti (congressi, matrimoni, cresime, ecc.) per conto dei clienti; ogni banchetto si tiene in un ristorante e prevede un certo menù.
- gli invitati ai singoli banchetti (ogni invitato ha un codice all'interno del banchetto);
- un elenco di ristoranti con le loro caratteristiche (nome, località, numero posti). I ristoranti offrono un certo insieme di menù, ciascuno costituito da portate. Ogni menù ha un codice all'interno del ristorante che lo offre; delle portate interessa il nome e il tipo (pesce, pasta, ecc.).

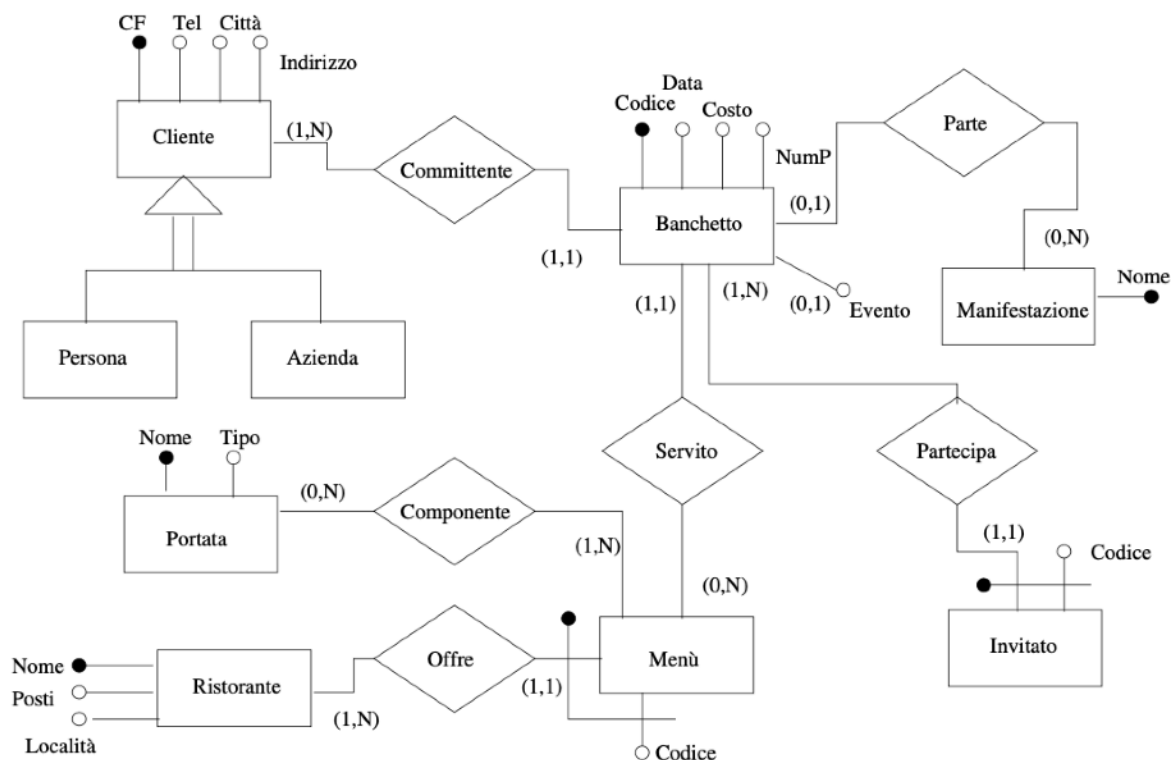
Progettare schema ER.

Soluzione.



Il testo dice che "ogni menù ha un codice all'interno del ristorante che lo offre", quindi il Codice del MENÙ non è un identificatore assoluto, ma un identificatore parziale, come potete vedere.

Riporto, per completezza, la soluzione proposta dal testo:



Esercizio 10 - Progettazione schema E/R con notazione Chen.

L'applicazione a cui si fa riferimento riguarda i concorsi pubblici, ed è descritta dai seguenti requisiti. Ogni concorso è identificato da un codice, prevede un certo numero di prove, un certo numero di vincitori, ed è indetto mediante un bando. Un bando è relativo a uno o più concorsi, ha una data di pubblicazione, un codice e una data di scadenza per la presentazione delle domande di ammissione. Per ogni concorso si nomina una commissione, formata da un certo numero di membri, uno dei quali è presidente, ed un altro è segretario. Dei membri della commissione interessa: codice fiscale, nome cognome, indirizzo e numeri di telefono. Al bando del concorso rispondono i candidati, dei quali interessa: codice fiscale, nome e cognome. Un concorso prevede una o più prove, ciascuna in una certa data, e con un certo punteggio massimo. Dei candidati che si presentano alle prove interessa anche l'indirizzo e il numero di telefono. Ogni candidato che effettua una prova totalizza un certo punteggio per quella prova. Alla conclusione del concorso viene stilata la

graduatoria, nella quale ogni candidato che si è presentato a tutte le prove compare con il relativo punteggio totale, ottenuto come somma dei punteggi che ha totalizzato alle prove.

Progettare schema ER.

Soluzione.

Di seguito viene riportata la mia soluzione.

Dal testo si legge "Per ogni concorso si nomina una commissione", il che normalmente fa pensare a un rapporto 1-a-1 (ogni concorso ha esattamente una commissione distinta). Se invece si volesse ammettere che la stessa commissione valga per più concorsi, allora 1-(N) avrebbe senso. In generale, se il requisito non lo specifica, la forma più frequente è 1-a-1.

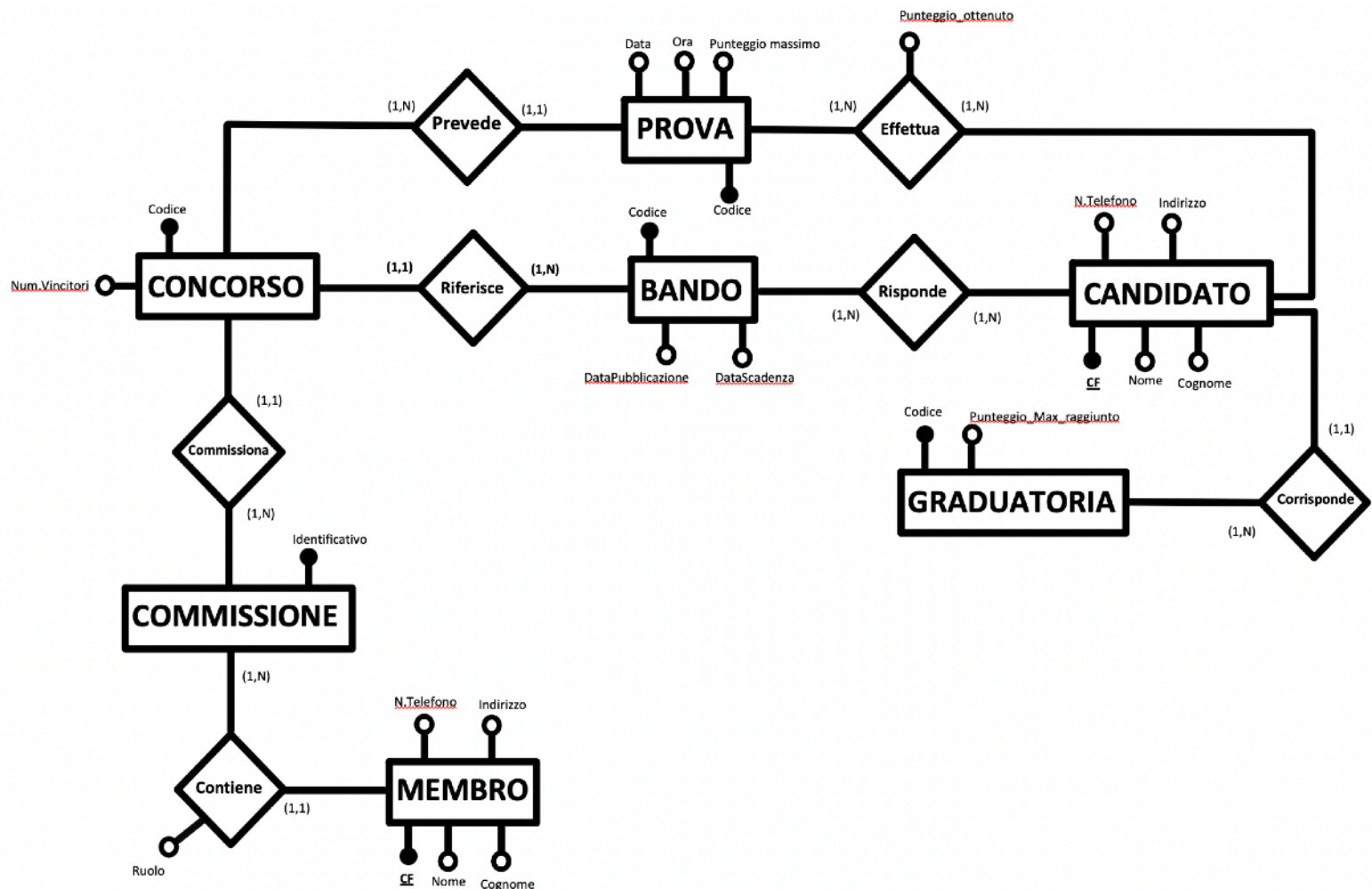
Talvolta, nei concorsi pubblici, la regola è che ogni concorso abbia la **propria** commissione (quindi sarebbe un 1-a-1), ma se le specifiche o la realtà dell'applicazione ammettono il riuso di una commissione per più concorsi, la mia scelta è coerente.

Il Numero_membri della **Commissione** si può calcolare contando i **Membri** legati da "Contiene".

Se "NumProve" corrisponde strettamente al conteggio delle entità **Prova** collegate a un concorso, sarebbe un attributo derivabile. Tuttavia se nel bando è fissato a priori il numero di prove che verranno svolte (a prescindere da quante prove poi *effettivamente* si istanzino), lasciarlo come attributo è ragionevole. Ma la specifica su questo non è chiara, quindi ho omesso questo attributo in **CONCORSO**.

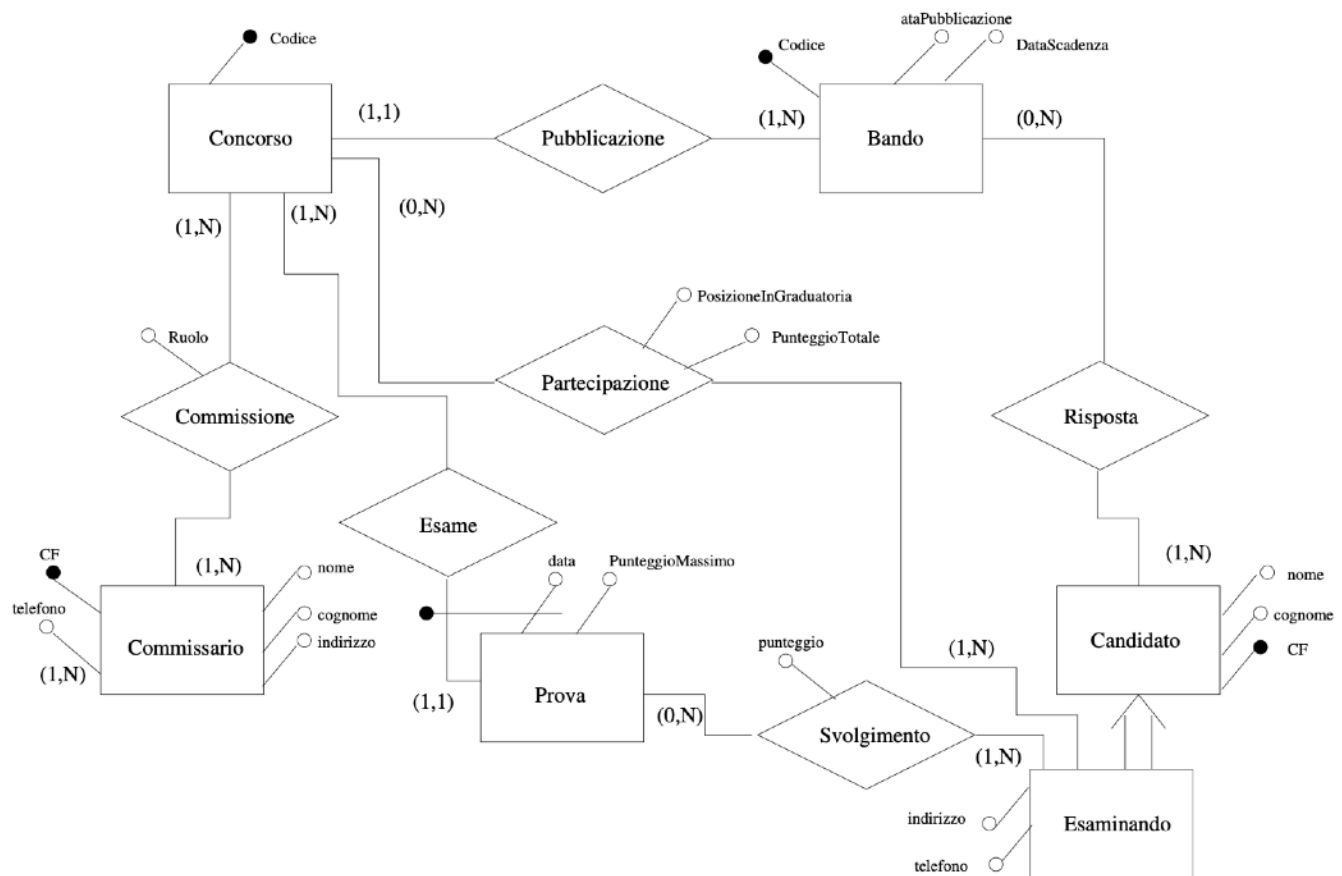
Ricorda di riportare solo le informazioni atomiche nello schema ER.

Due paroline sugli attributi di relazione "**punteggio_ottenuto**" su Effettua e "**Ruolo**" su Contiene. Entrambi gli attributi dipendono strettamente dall'associazione fra le entità, e non dall'entità in sé; per questo vanno correttamente sulla relazione.



Di seguito la soluzione della rivista dove ho preso il testo.
 Nella soluzione proposta dal testo si è distinto tra il candidato che risponde al bando e l'esaminando che si presenta alle prove.

- La relazione Risposta è ridondante in quanto si può risalire ai bandi a cui un candidato ha risposto dai concorsi a cui partecipa che siano pubblicati in quel bando;
- Gli attributi PosizioneGraduatoria e PunteggioTotale sono ridondanti in quanto possono essere generati attraverso i punteggi ottenuti dai candidati nelle varie prove.



Esercizio 11 - Progettazione schema E/R con notazione Chen.

Definire uno schema Entità-Relazione che descriva i dati di un'applicazione per gestire un'agenzia matrimoniale. Le informazioni di interesse riguardano:

I clienti: uomini e donne, di cui interessa il numero di tessera, il nome (inteso come nome e cognome), l'età, l'aspetto fisico (piacevole, interessante, bello, ...), l'insieme dei suoi interessi (es. cinema, arte, vela, ...), ed eventualmente la sua professione. I clienti si dividono (oltre che in uomini e donne) anche in **clienti regolari** e **clienti in prova**.

Le opzioni: un cliente regolare può dichiarare all'agenzia il suo interesse per uno o più clienti dell'altro sesso.

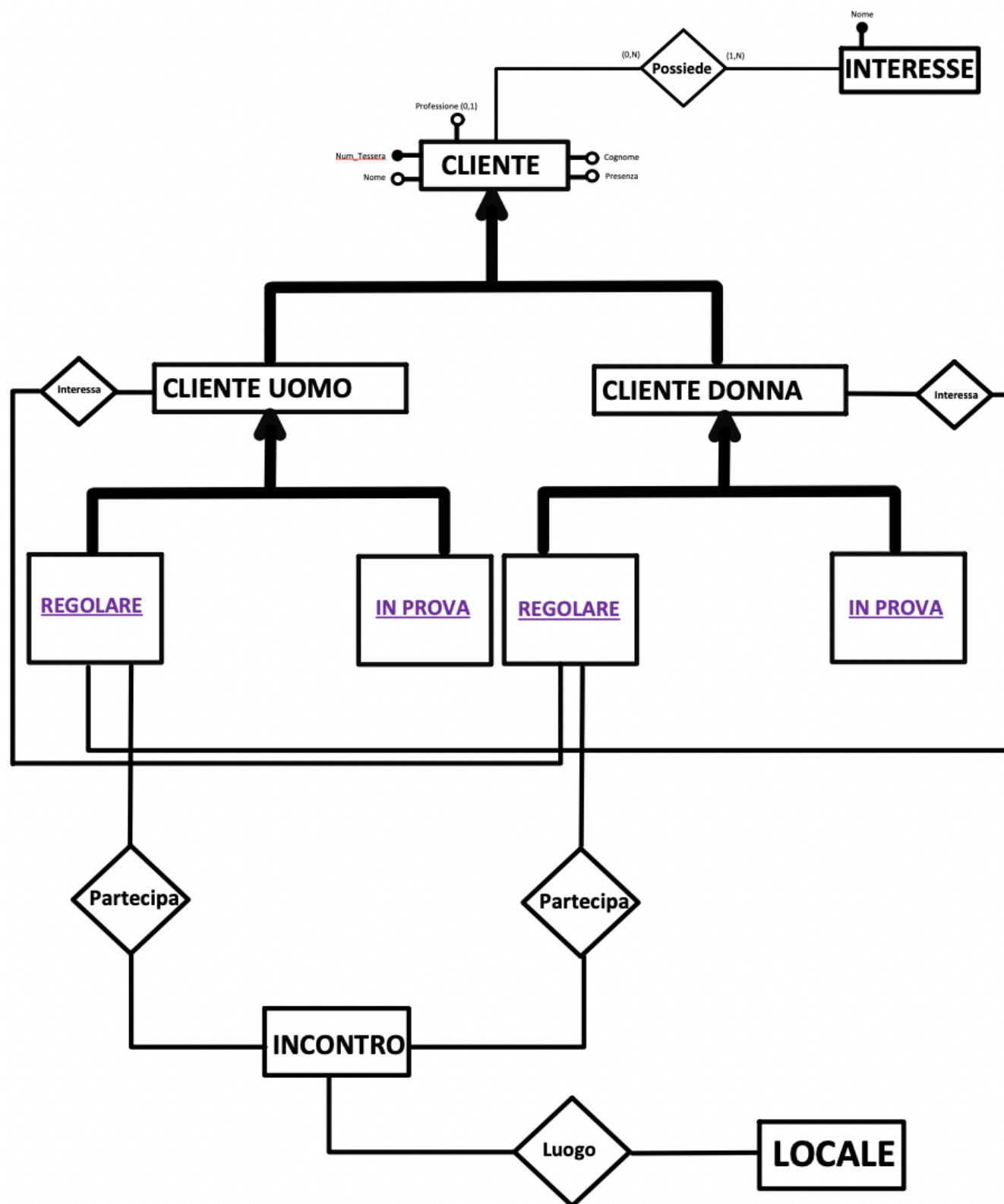
Gli incontri: L'agenzia programma incontri tra clienti regolari. Per ogni incontro interessano i due partecipanti, la data, il locale in cui si è svolto, e l'eventuale esito (positivo o negativo). Due clienti possono avere anche più incontri (in date diverse).

I locali: Ristoranti convenzionati con l'agenzia, di cui interessa

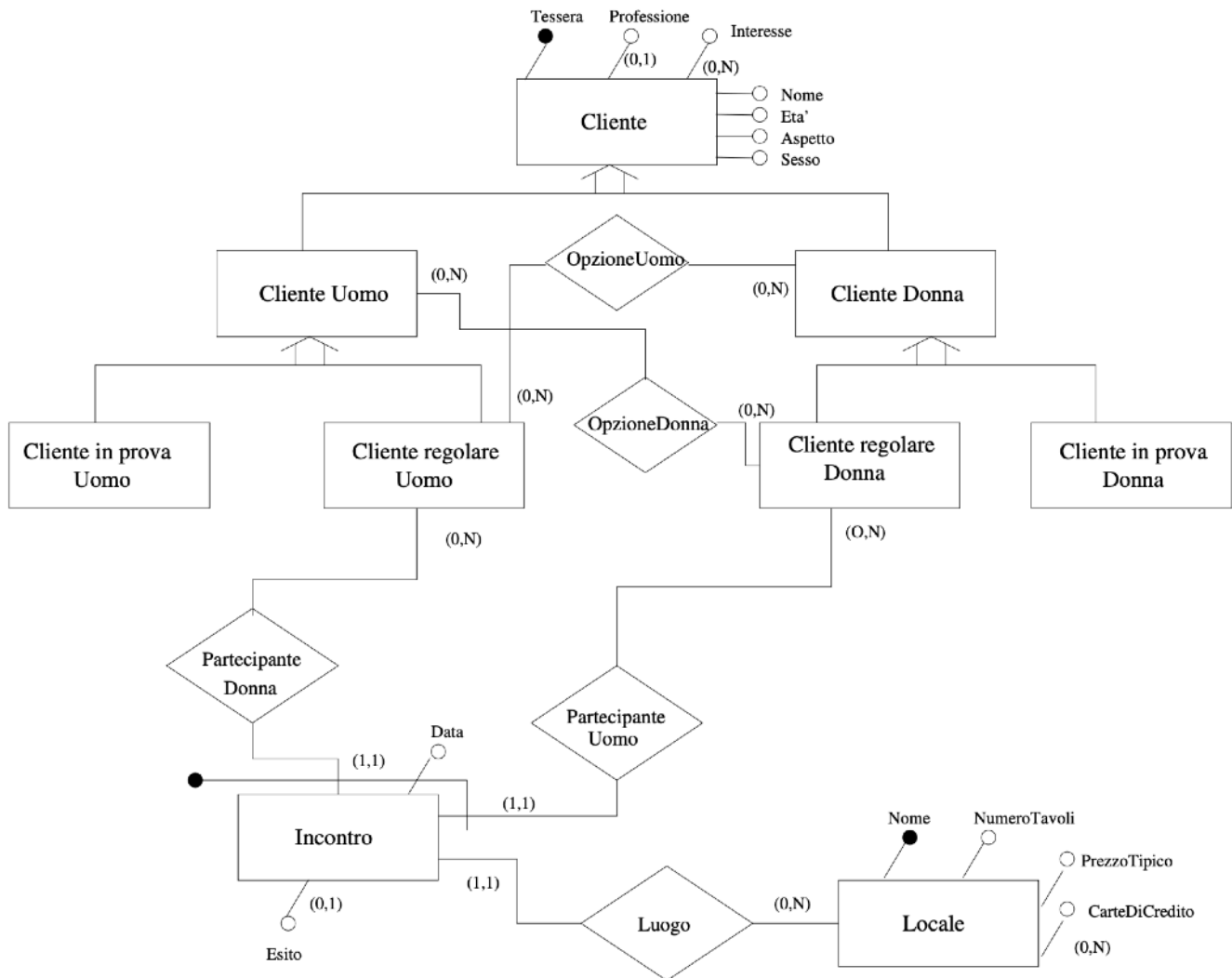
il nome, il numero di tavoli, il prezzo tipico e le carte di credito accettate.

Progettare schema ER.

Soluzione.



Questa è la mia soluzione abbozzata al volo, di seguito la soluzione fornita dal testo:



Io ho una tabella di interessi, perché mi piace tenere traccia di tutti gli interessi.

